NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Computer Networks

Redes de Computadores

RC

Teórica CINCO

LEI

João Leitão

©2026 NOVA FCT

Aula Cinco

- Recapitulação Rápida: O Que Faz o Transporte, e Portas
- TCP vs. UDP: Visão Geral
- User Datagram Protocol (UDP)
- Porque Escolher UDP em vez de TCP?
- O Problema da Fiabilidade: ARQ e Stop-and-Wait
- O Problema de Eficiência do Stop-and-Wait

Redes de Computadores

- Nas últimas aulas temos estudado o que uma rede precisa de fornecer, e como avaliar se o está a fazer bem.
- A partir de hoje: dado um serviço de entrega de pacotes best-effort entre hosts (fornecido pela camada de rede – L3), como construímos algo sobre isso em que um programador de aplicações possa efetivamente confiar?

Transporte

Conclusão:
este é o nível de transporte, também chamado de protocolo fim-a-fim — é o que finalmente liga um processo de aplicação a outro, e não apenas uma máquina a outra.

Projeto 1

Conclusão:
O Projeto 1 é construído inteiramente sobre o protocolo que abordamos hoje (UDP).

Revisão: O Que Faz o Transporte

- Comunicação processo-a-processo, construída sobre a entrega best-effort da rede entre hosts.
- **O que uma aplicação poderia razoavelmente querer:** entrega garantida, ordenada e exatamente uma vez; mensagens arbitrariamente grandes; sincronização; controlo de fluxo; multiplexagem.
- **O que a rede realmente oferece, por baixo:** nada disso é garantido — as mensagens podem ser perdidas, reordenadas, duplicadas, limitadas em tamanho, ou arbitrariamente atrasadas.
- Diferentes protocolos de transporte fecham essa diferença em graus diferentes. O UDP não fecha quase nada dela — e é exatamente esse o ponto de hoje.

Revisão: Portas e Desmultiplexagem

- Vários processos podem partilhar um host e um endereço IP — um pacote que chega precisa de alcançar o processo certo.
- **O mecanismo:** a porta — um localizador abstrato, por host. Um processo é na realidade identificado por um par (host:porta).
- **Portas bem conhecidas:** DNS na porta 53, submissão de correio na porta 25 — publicadas para que um cliente saiba para onde enviar a primeira mensagem.

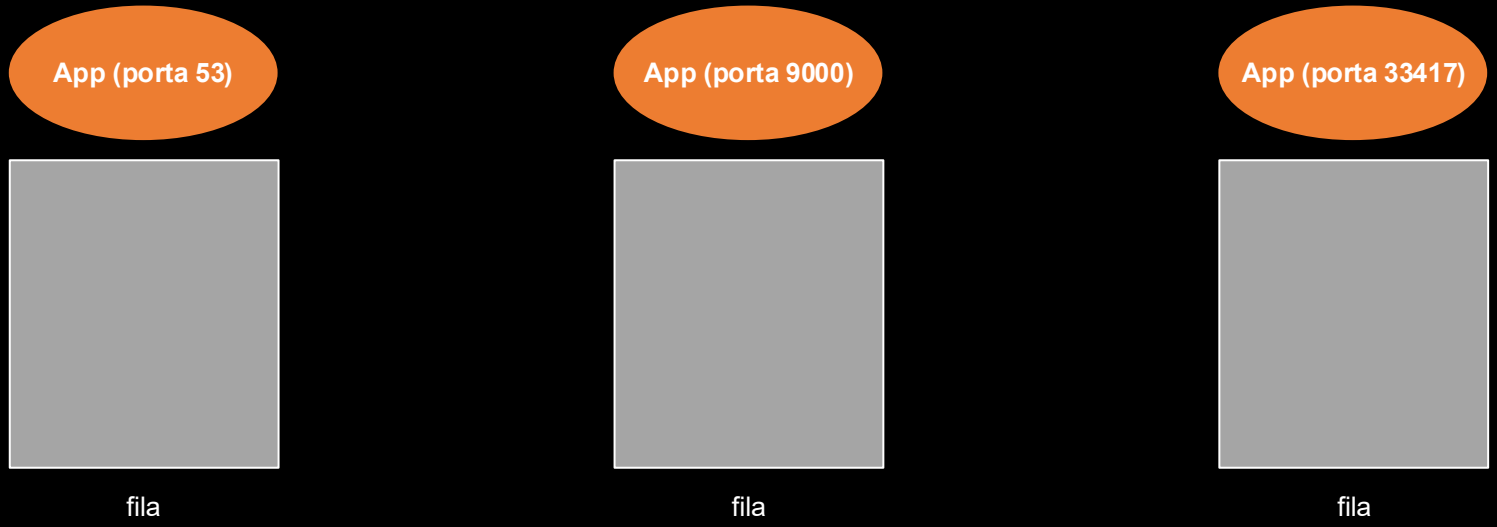
Desmultiplexagem em Ação

App (porta 53)

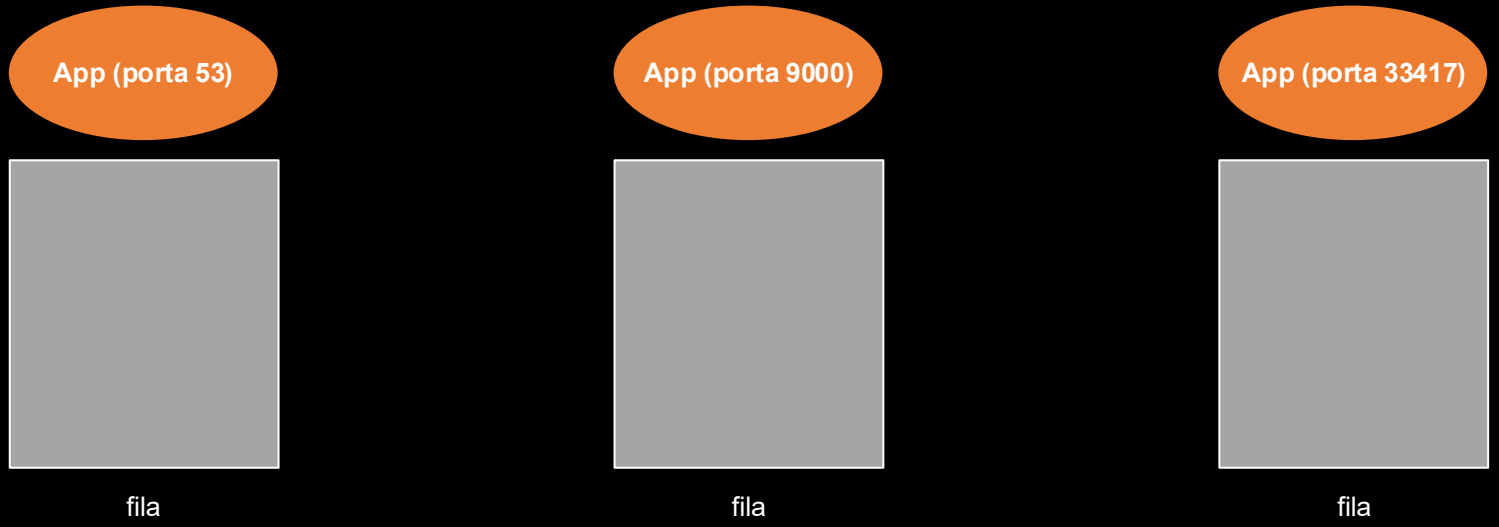
App (porta 9000)

App (porta 33417)

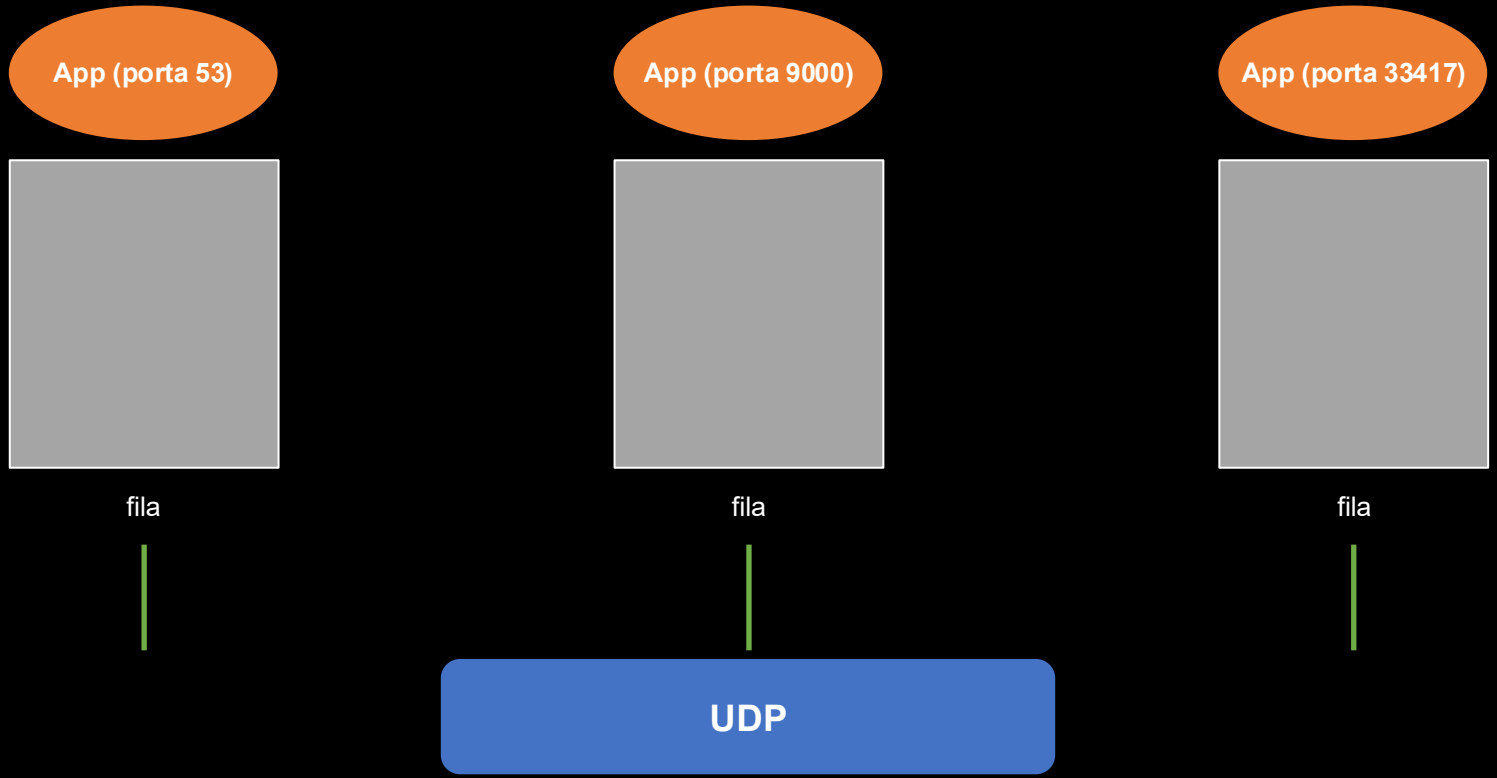
Desmultiplexagem em Ação



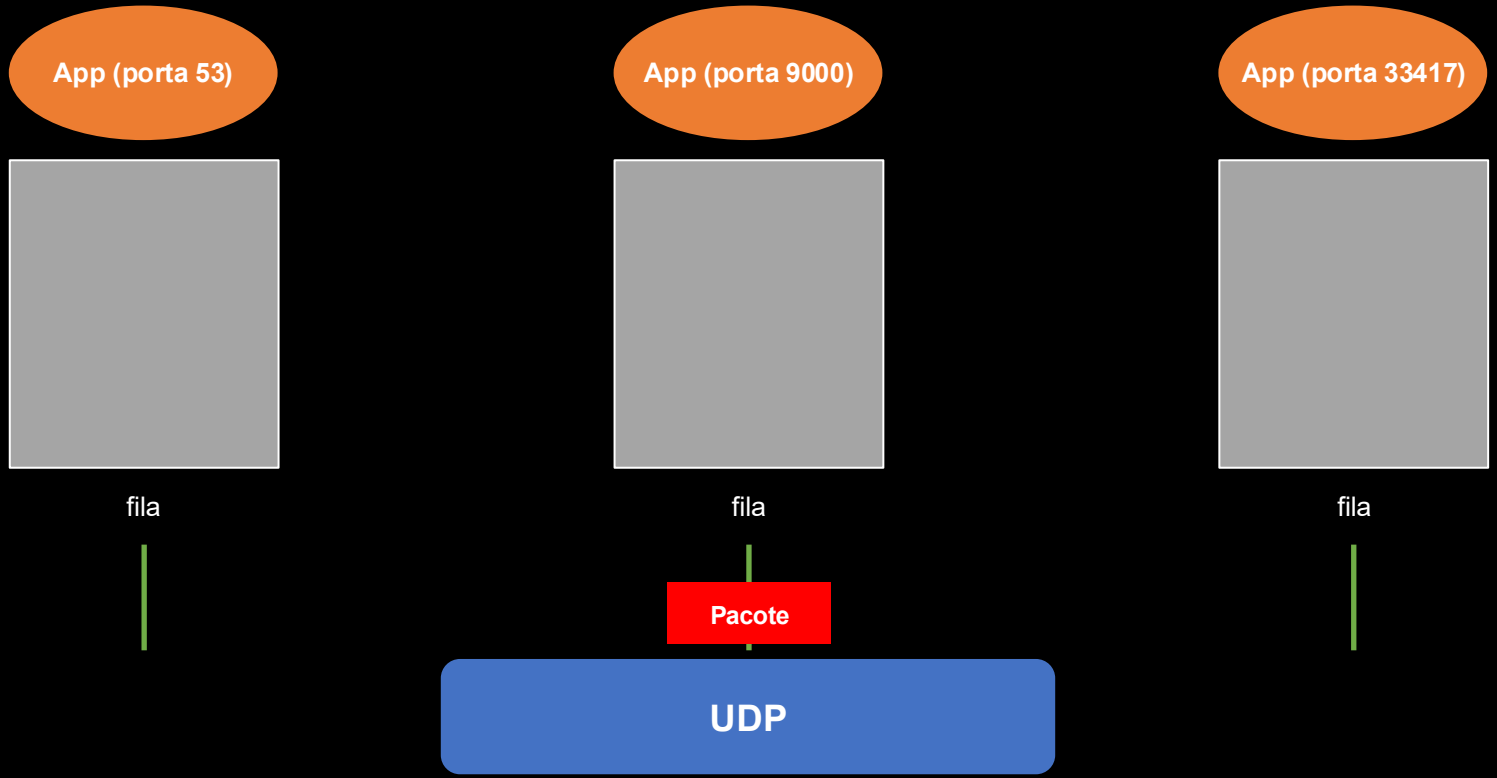
Desmultiplexagem em Ação



Desmultiplexagem em Ação

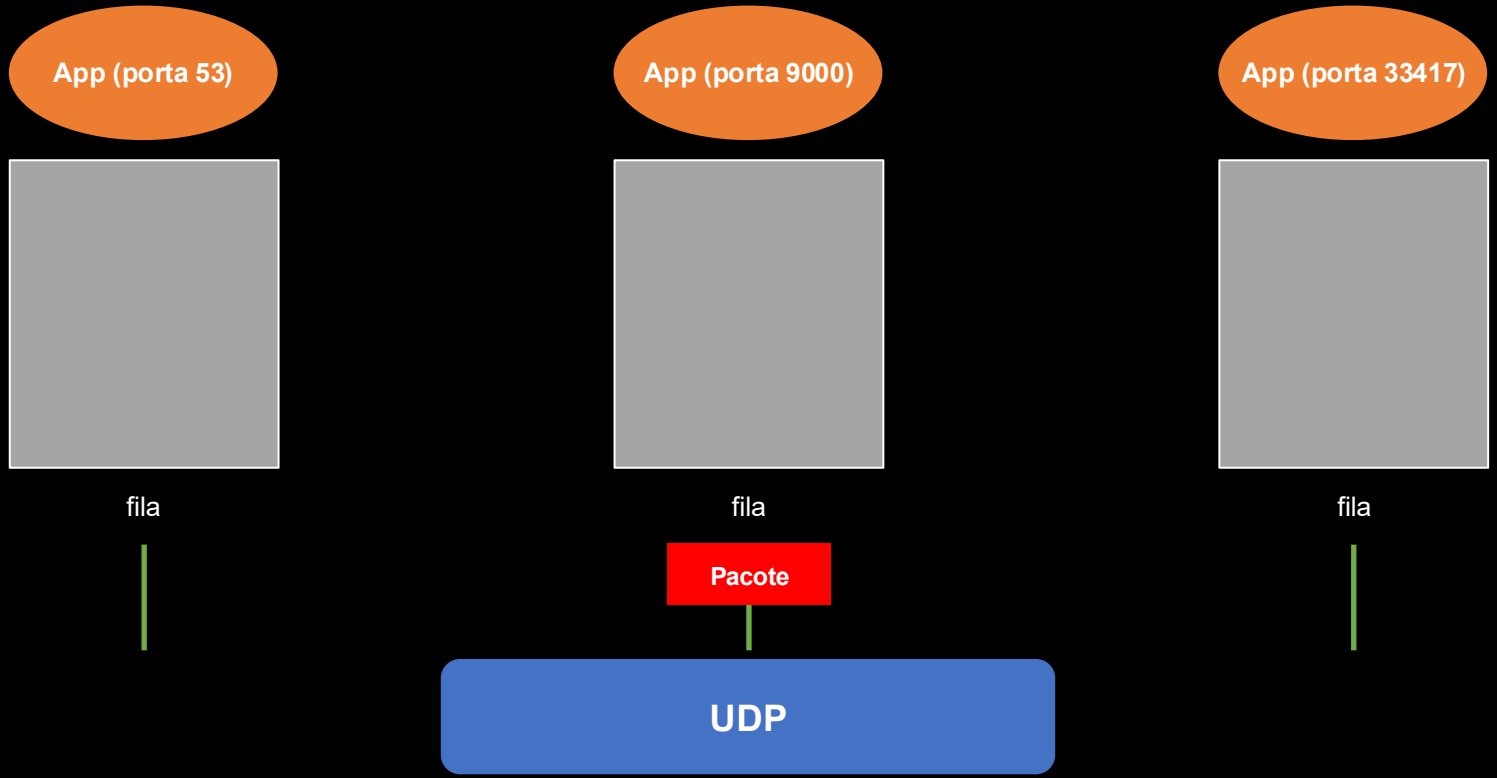


Desmultiplexagem em Ação



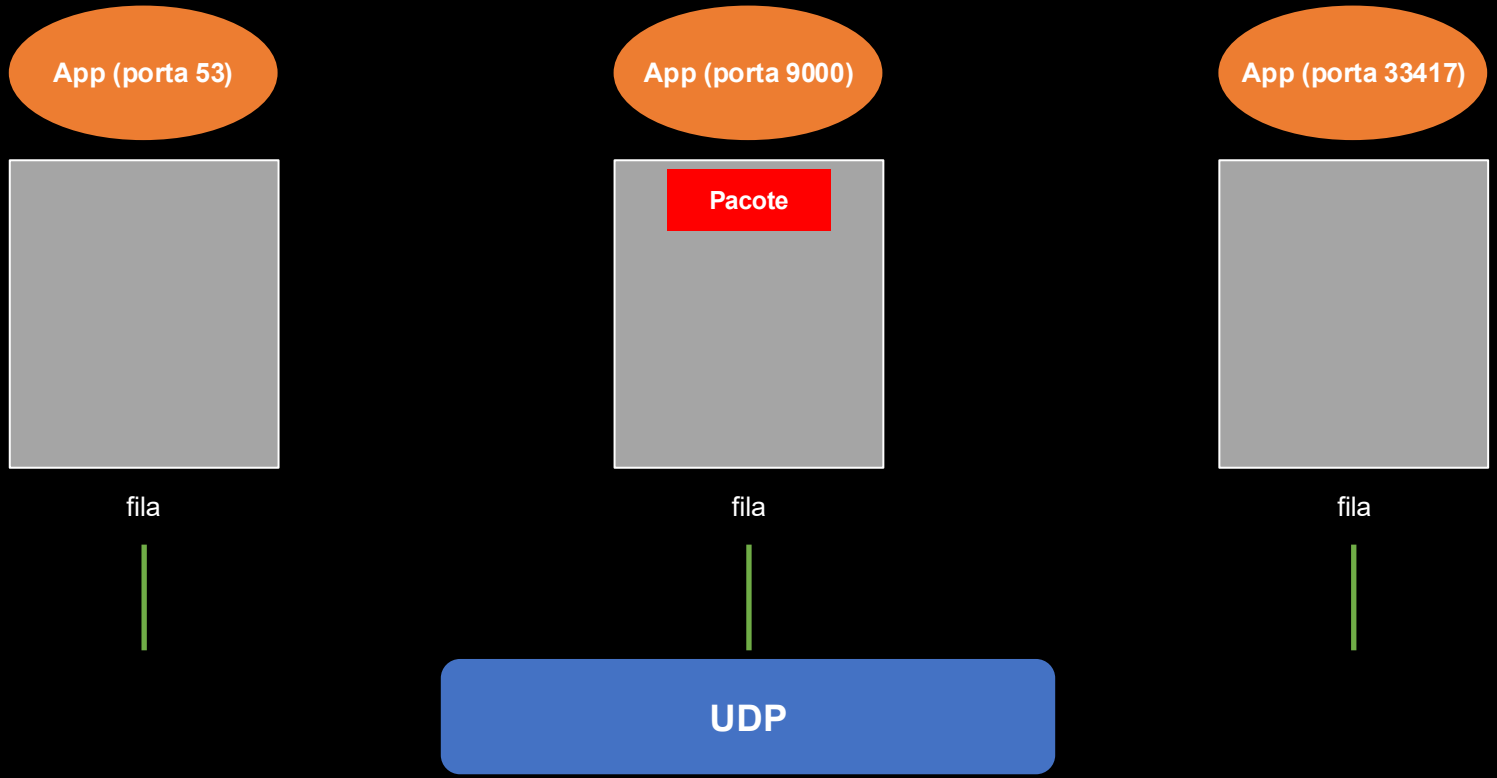
Chega um pacote, porta de destino = 9000.

Desmultiplexagem em Ação



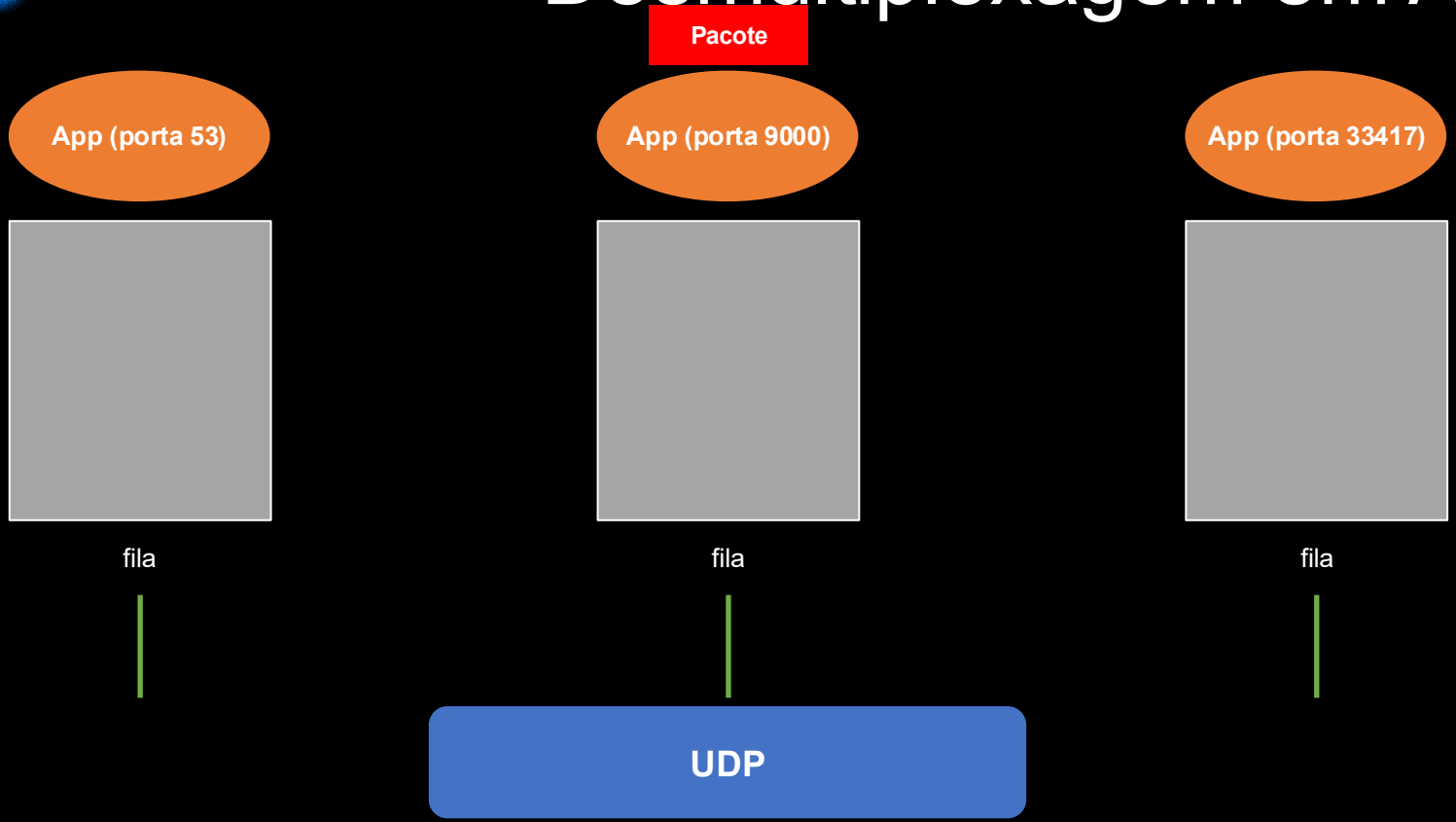
O UDP verifica a porta de destino e encaminha-o para a fila correspondente.

Desmultiplexagem em Ação



O UDP verifica a porta de destino e encaminha-o para a fila correspondente.

Desmultiplexagem em Ação



Entregue. As filas das outras duas apps nunca foram tocadas.

TCP vs. UDP: Visão Geral

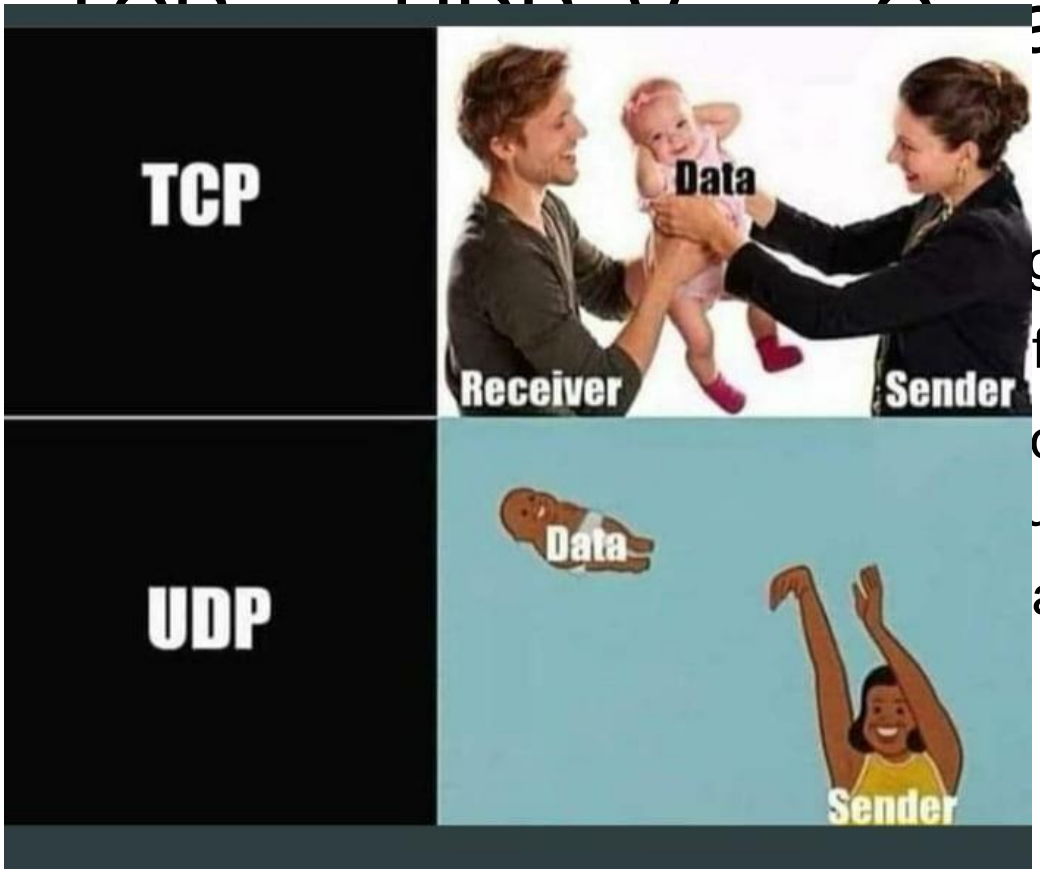
TCP

- Orientado a ligação
- Fiável, ordenado
- Controlo de fluxo + controlo de saturação
- Cabeçalho mais pesado (20+ bytes)

UDP

- Sem ligação
- Best-effort, não ordenado
- Sem controlo de fluxo ou de saturação
- Cabeçalho mínimo (8 bytes)

TCP vs UDP: Visão Geral



gação

ffort, não ordenado

ontrolo de fluxo ou
uração

alho mínimo (8

TCP e UDP: Visão Geral





TCP



UDP



Um Pouco de História (1/2)

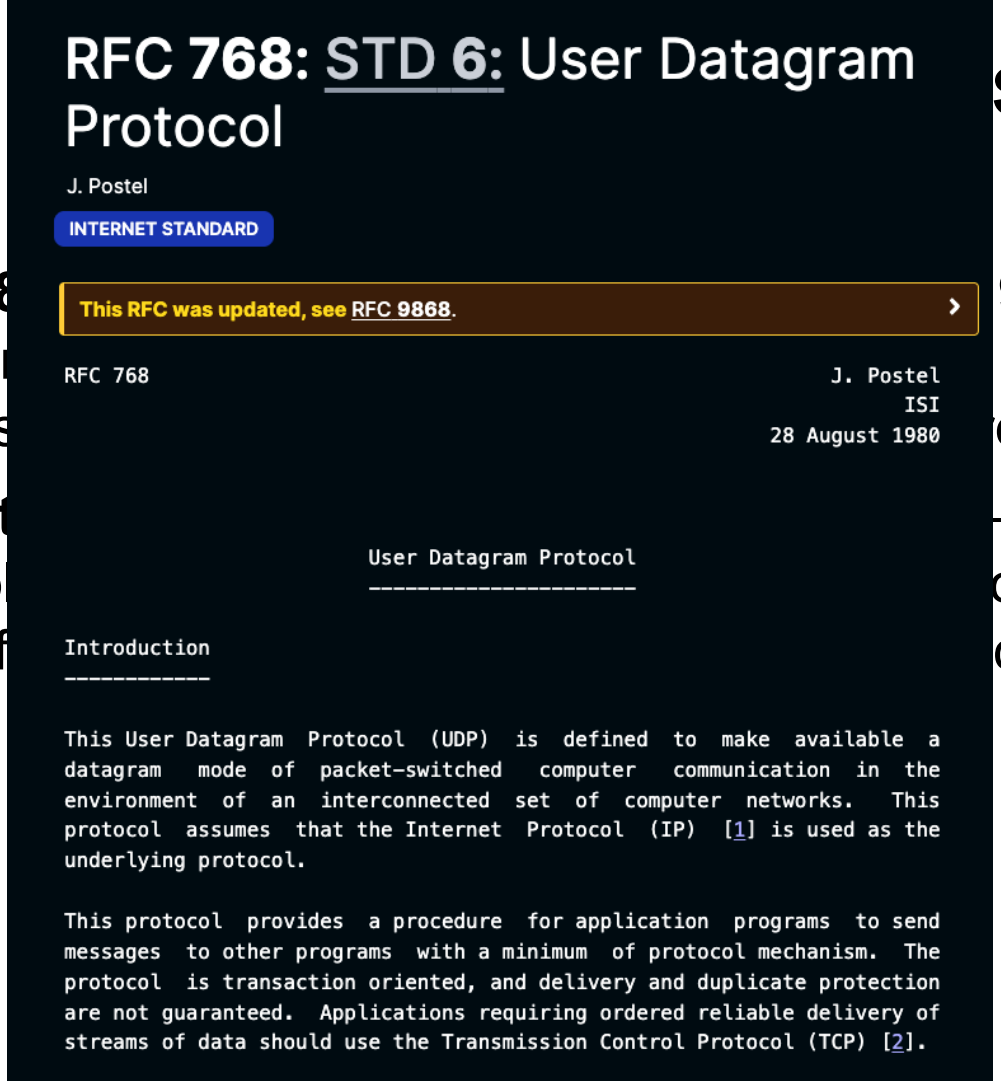
- **O RFC 768,** escrito por Jon Postel em agosto de 1980 — um dos primeiros protocolos de transporte da Internet, e um dos RFCs mais curtos alguma vez escritos (cerca de três páginas).
- **Minimalista por conceção,** não por negligência — o UDP é um exemplo precoce e elegante da filosofia “não colocar na rede uma funcionalidade de que nem todas as aplicações precisam.”

stória (1/2)

- **O RFC 768**
  dos primei
  RFCs mais
- **Minimalist**
  um exemp
  rede uma f
  precisam.”

980 — um
e um dos
ês páginas).

– o UDP é
olocar na
cações



RFC 768: STD 6: User Datagram Protocol

J. Postel

INTERNET STANDARD

This RFC was updated, see RFC 9868.

RFC 768

J. Postel
ISI
28 August 1980

User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Um Pouco de História (2/2)

- **Ainda hoje omnipresente:** DNS, DHCP, streaming e média em tempo real, VPNs — e o QUIC, o transporte por baixo do HTTP/3 moderno, está ele próprio construído sobre UDP.
- **Porque é que a Google se deu ao trabalho de reconstruir o transporte sobre UDP:** O QUIC reduziu de forma mensurável em cerca de um segundo no tempo de carregamento de páginas da Google Search nas ligações mais lentas, e reduziu o rebuffering do YouTube em cerca de 30%.

Garantias do UDP

- I would tell you a UDP joke…

Garantias do UDP

- I would tell you a UDP joke…
- …but you might not get it.

Garantias do UDP

- I would tell you a UDP joke…
- …but you might not get it.
- **Tem piada porque é verdade:** a ausência de garantias de entrega é um principio de design e não um bug.

O Cabeçalho UDP (1/2)

- A desmultiplexagem (demux) precisa que a porta de destino chegue efetivamente com o pacote — Como é que inserimos essa informação no cabeçalho (header) de um datagrama UDP?
- **SrcPort, DstPort —** 16 bits cada, até 64K portas por host (por interface de rede).
- **Length —** tamanho do cabeçalho + dados, em bytes.

O Cabeçalho UDP (2/2)

- **Checksum —** verificação de correção opcional (mais sobre isto mais à frente).
- Seguido de Data (os dados da aplicação) — o que quer que a aplicação tenha passado ao sendto().
- **Cabeçalho fixo total: 8 bytes —** substancialmente mais simple que o cabeçalho do TCP, de 20+ bytes.

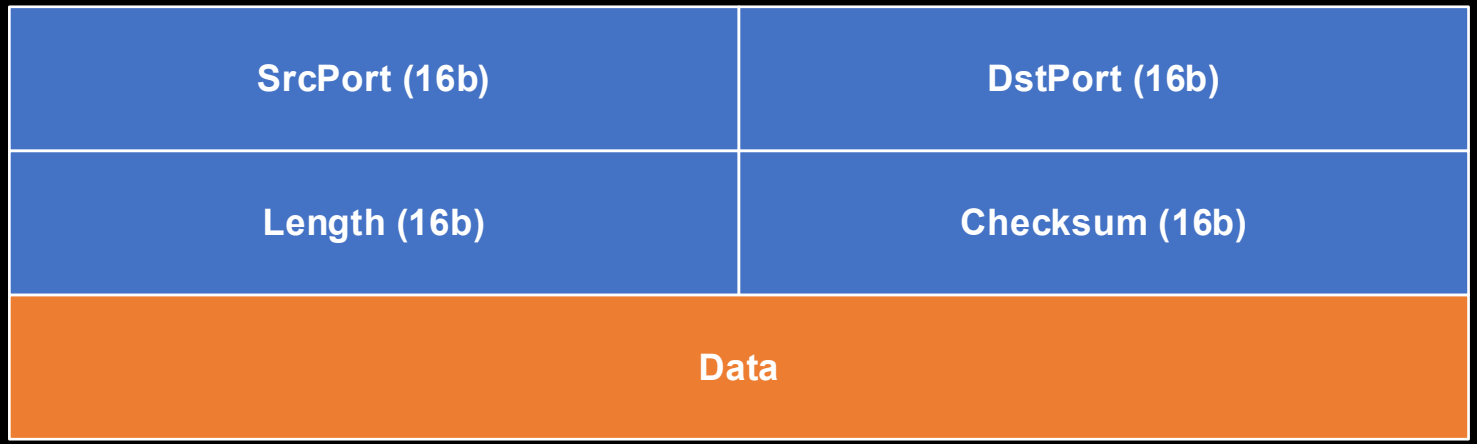
Estrutura do Cabeçalho UDP



Estrutura do Cabeçalho UDP

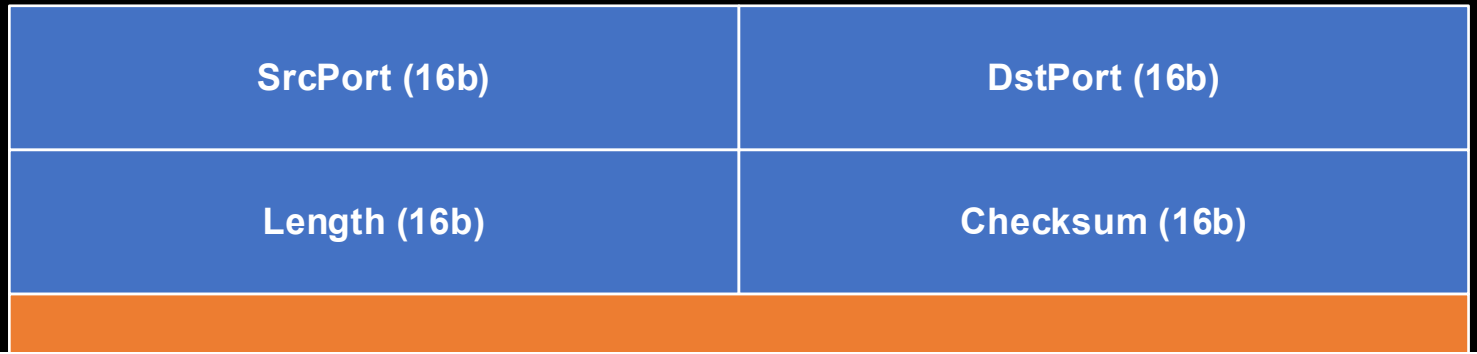
SrcPort (16b)	DstPort (16b)
Length (16b)	Checksum (16b)

Estrutura do Cabeçalho UDP



Estrutura do Cabeçalho UDP

Cabeçalho UDP: 4 campos, cada um com 2 bytes. 8 bytes no total.



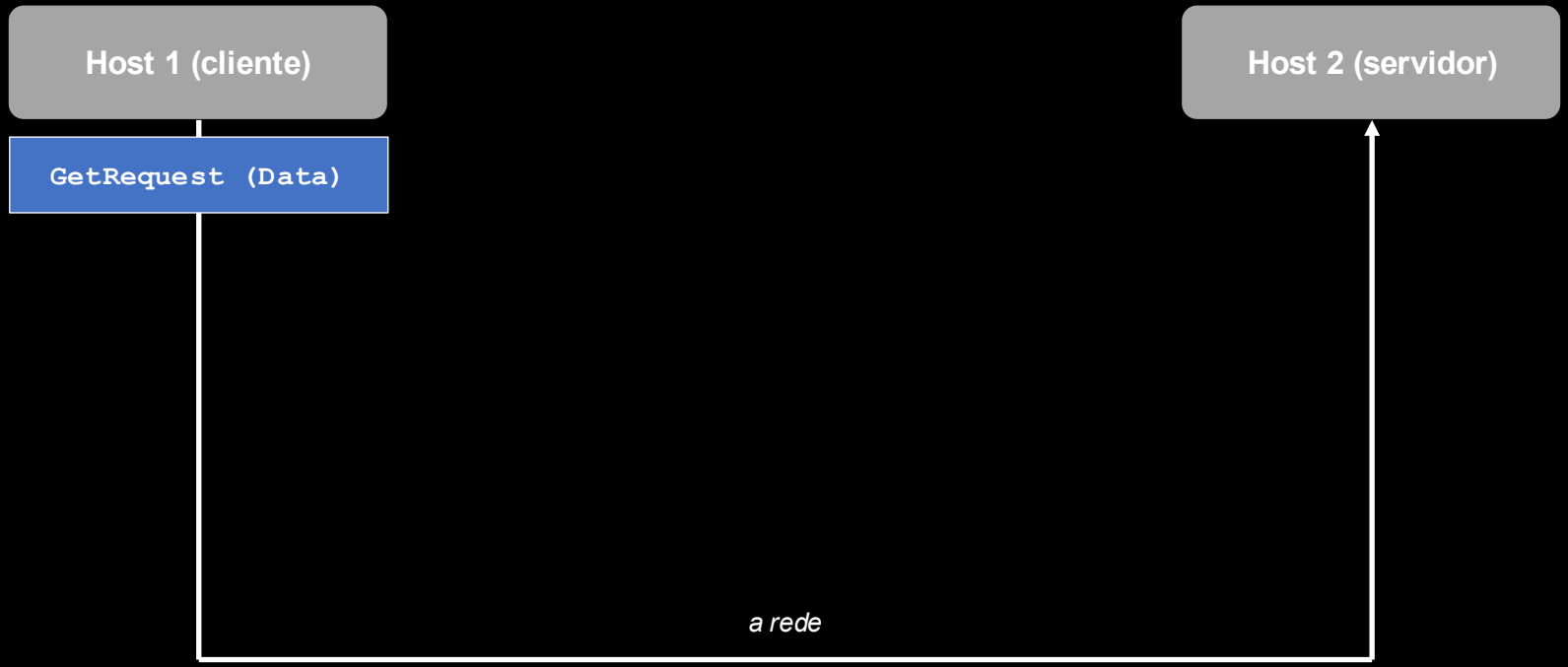
IPv4 pseudo header format

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source IPv4 Address																															
4	32	Destination IPv4 Address																															
8	64	Zeroes								Protocol								UDP Length															
12	96	Source Port																Destination Port															
16	128	UDP Length																Checksum															
20	160+	Data																															

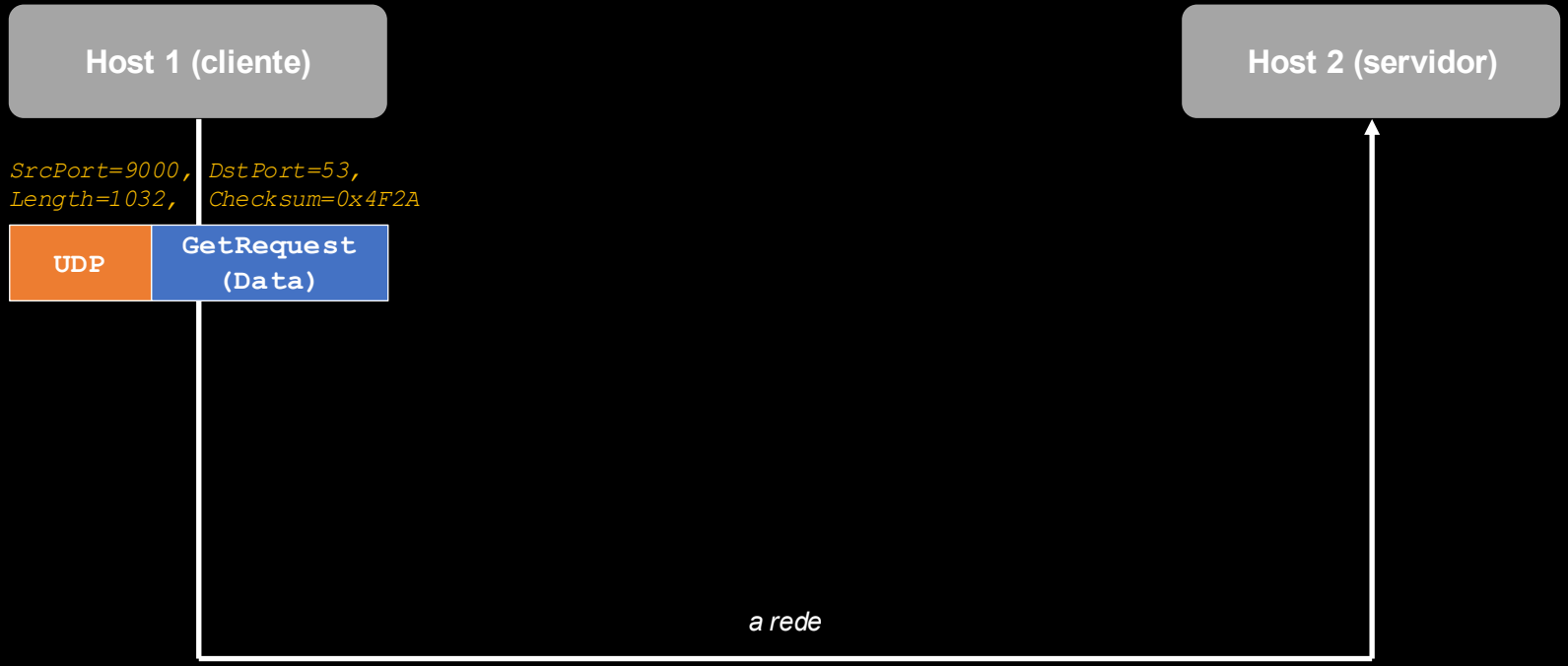
Ligação à Teórica 2

- Quando um cliente chama o método sendto(), os bytes que lhe são entregues não são o que efetivamente sai da máquina para a rede. O que é acrescentado, e por que ordem, antes de chegar ao meio físico — e o que é retirado, por que ordem, do outro lado?

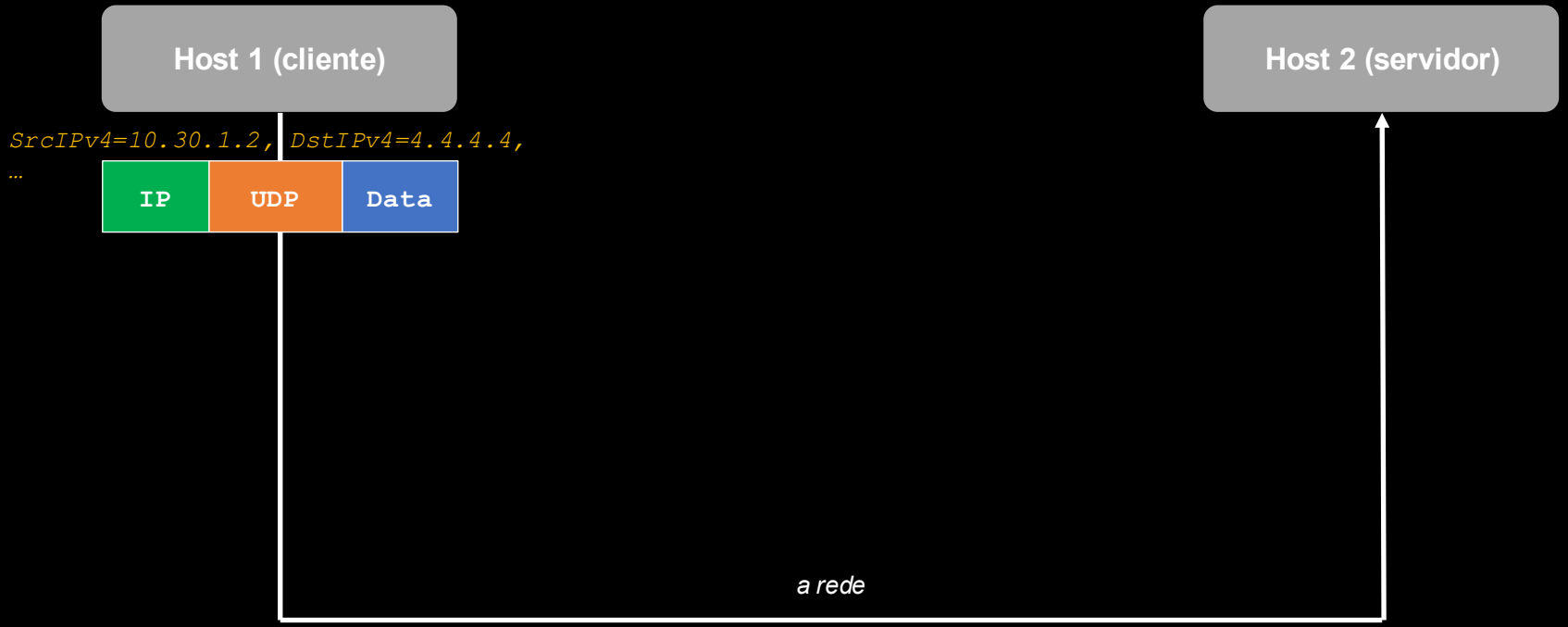
Encapsulação, Materializada



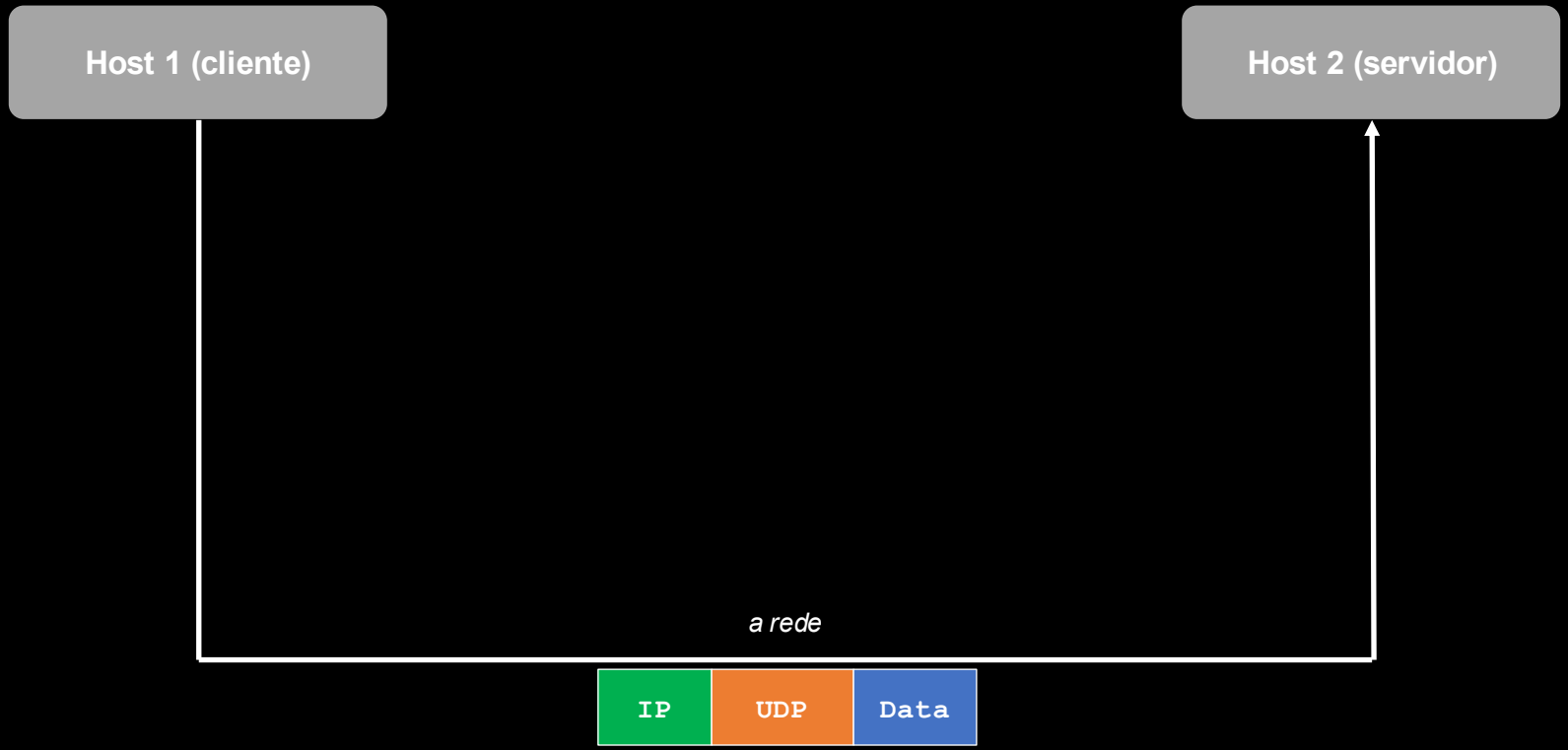
Encapsulação, Materializada



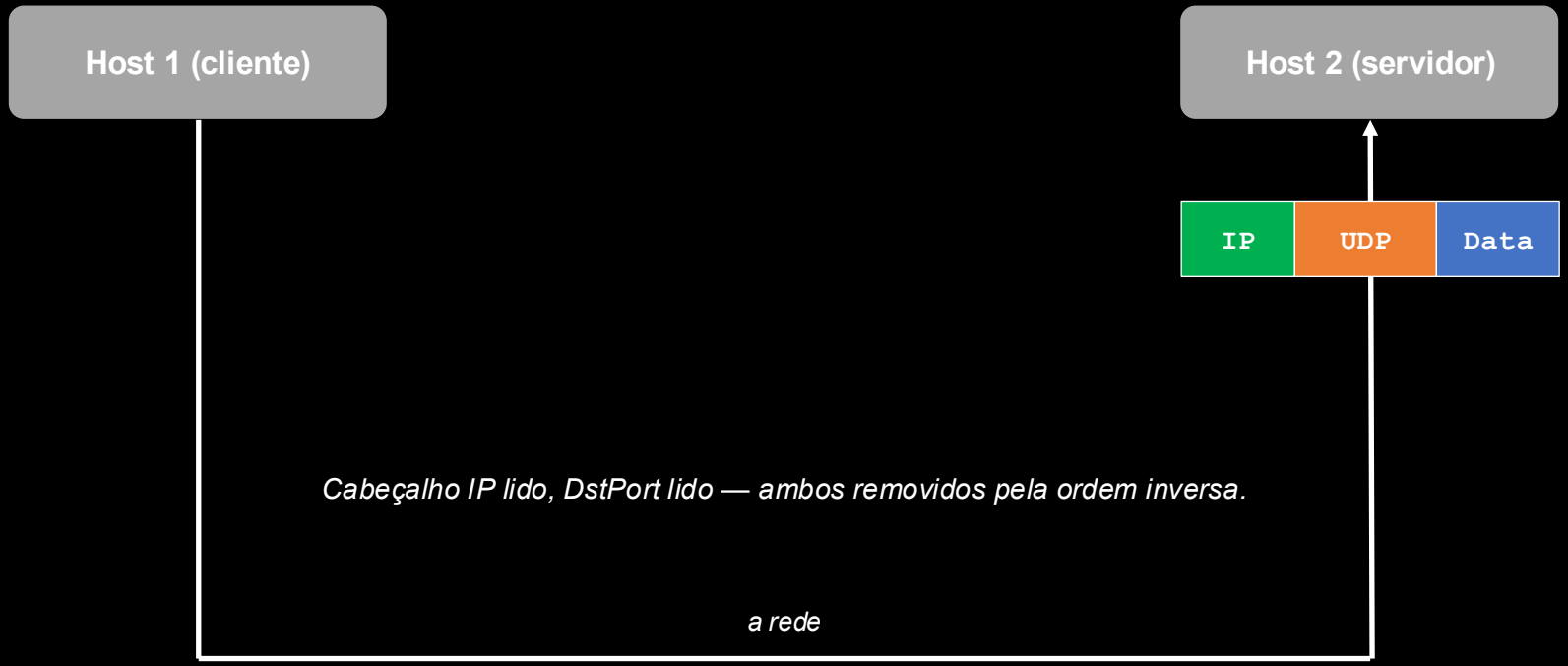
Encapsulação, Materializada



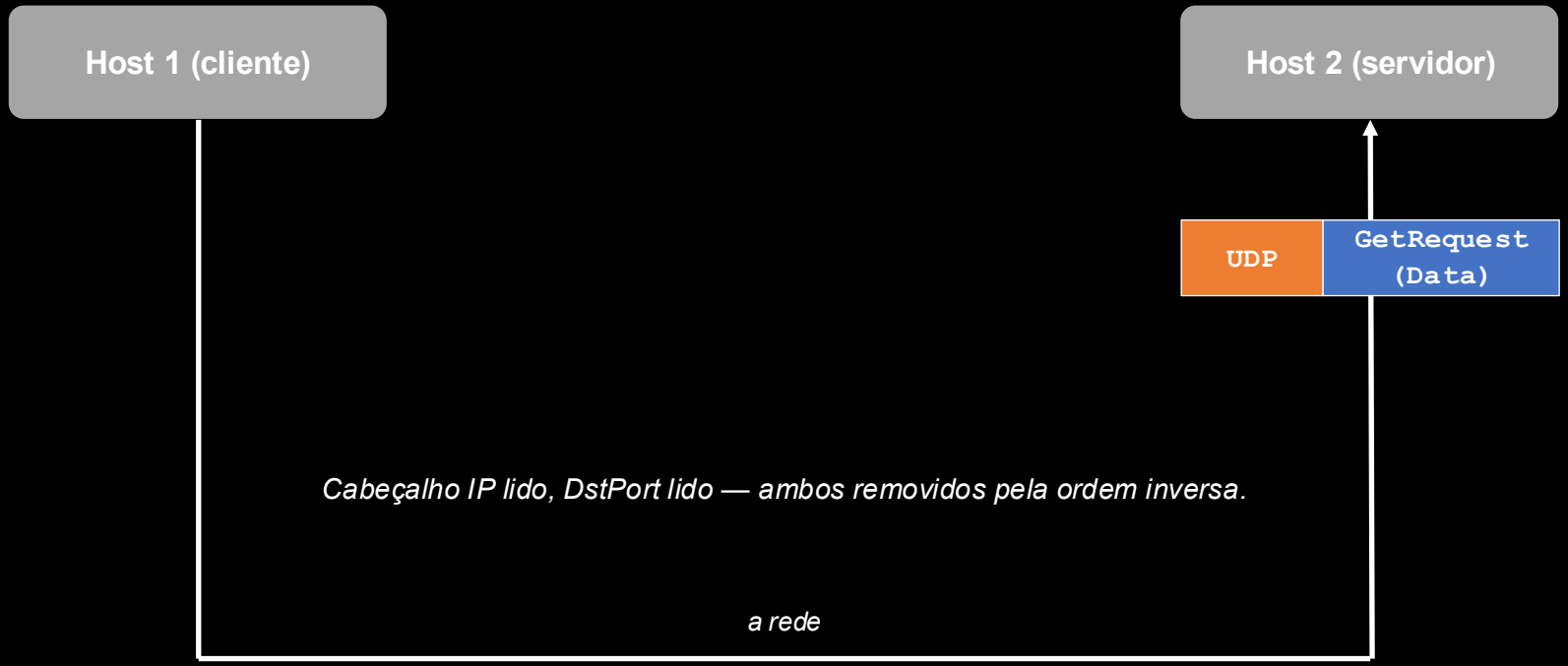
Encapsulação, Materializada



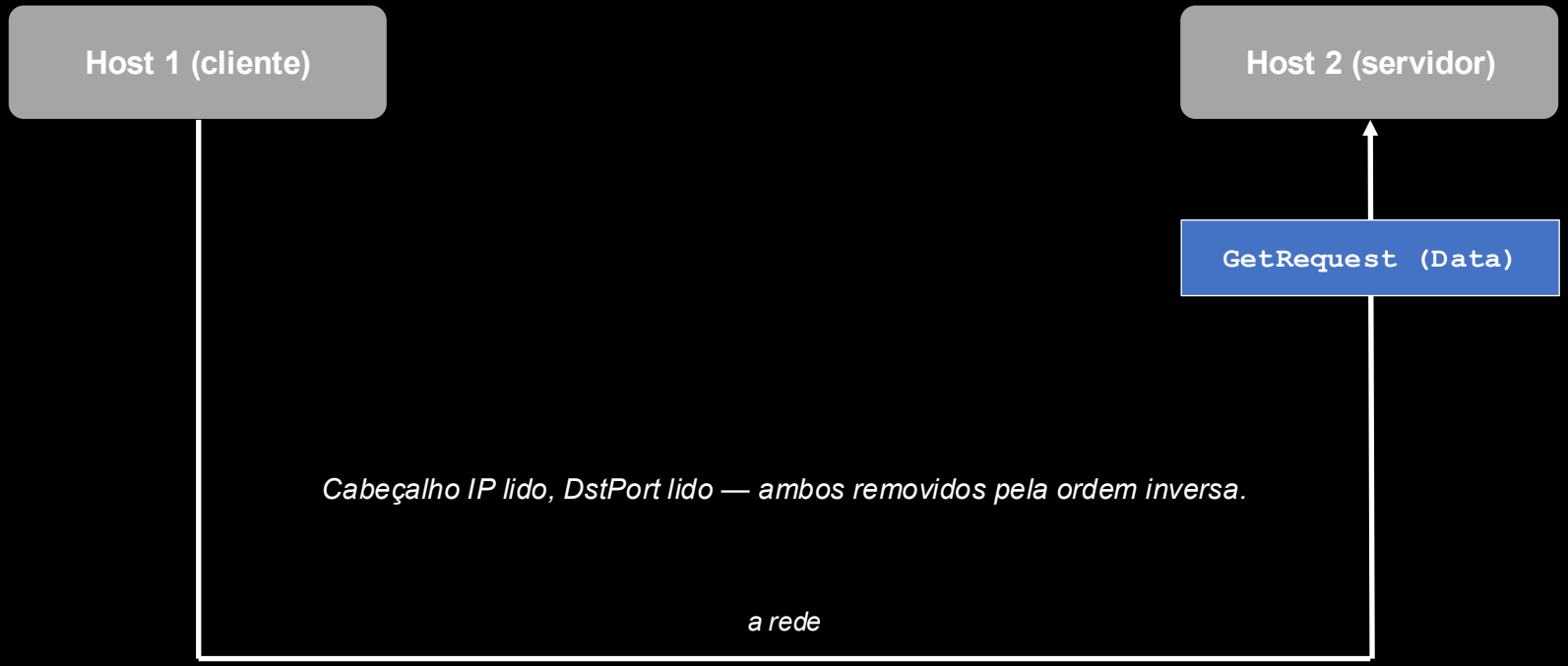
Encapsulação, Materializada



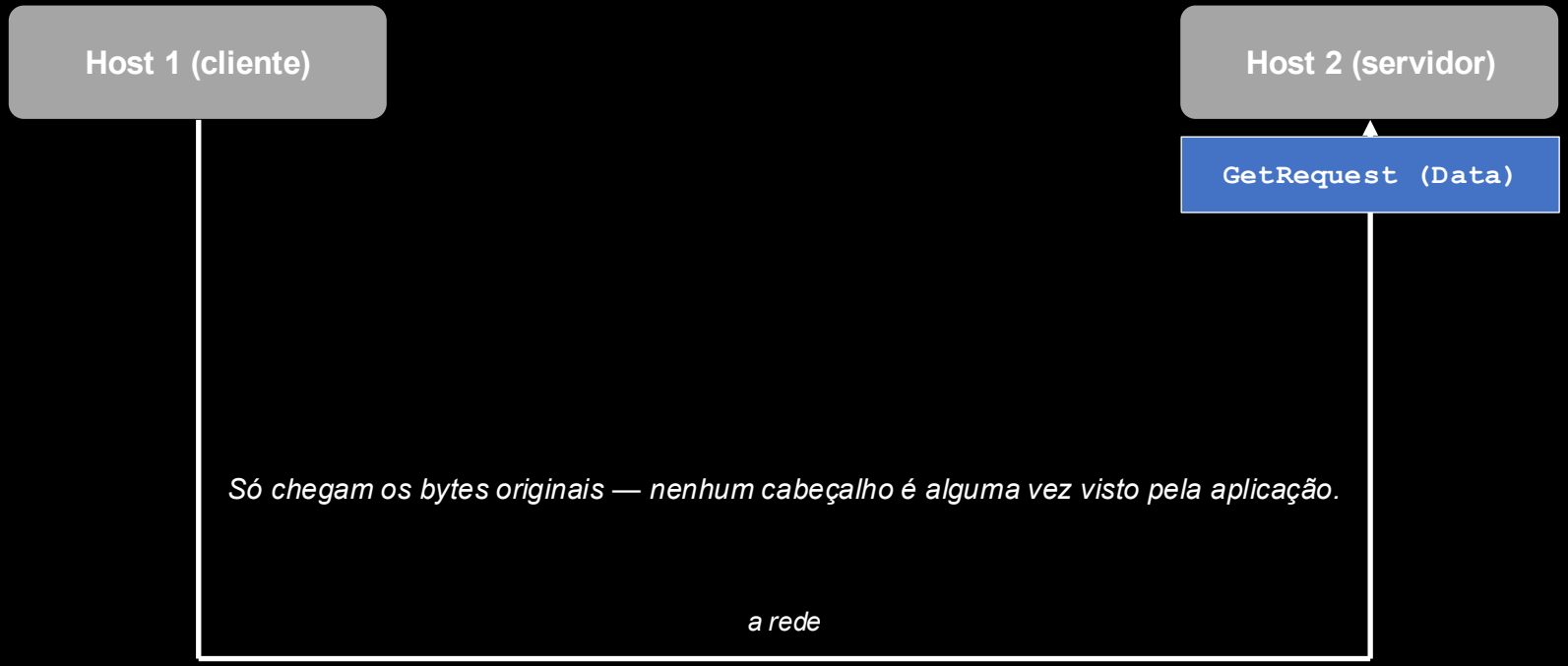
Encapsulação, Materializada



Encapsulação, Materializada



Encapsulação, Materializada



Já vimos isto antes…

Conclusão:
isto é exatamente o mesmo comportamento que vimos na Teórica 2 para HTTP-sobre-TCP-sobre-IP — basta trocar o pedido HTTP do browser por um pedido de outra aplicação, e o TCP por UDP.

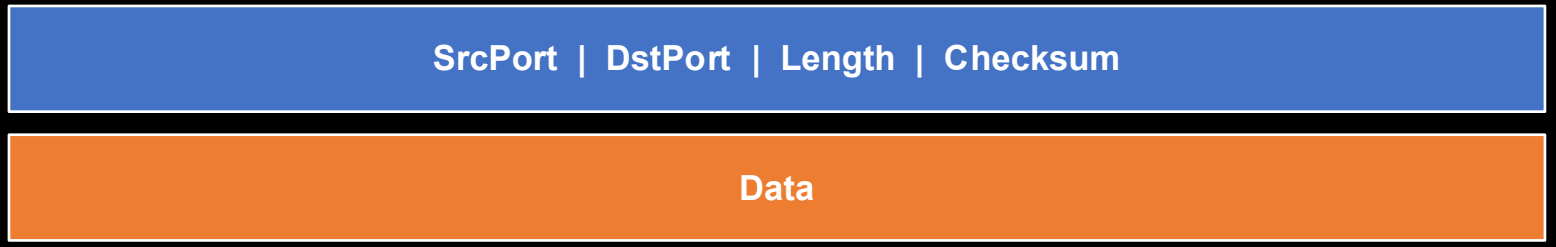
O Checksum (1/2)

- A única outra tarefa do UDP além da multiplexagem/desmultiplexagem: uma verificação de correção opcional (em IPv4).
- O checksum no UDP permite ao recetor verificar se o conteúdo do datagrama foi recebido corretamente (i.e., se não houve corrupção de dados em trânsito).
- Vamos supor que o endereço IP de destino é corrompido em trânsito, e o seu pacote é entregue ao host errado. Será que o UDP sequer consegue detetar isso?

O Checksum (2/2)

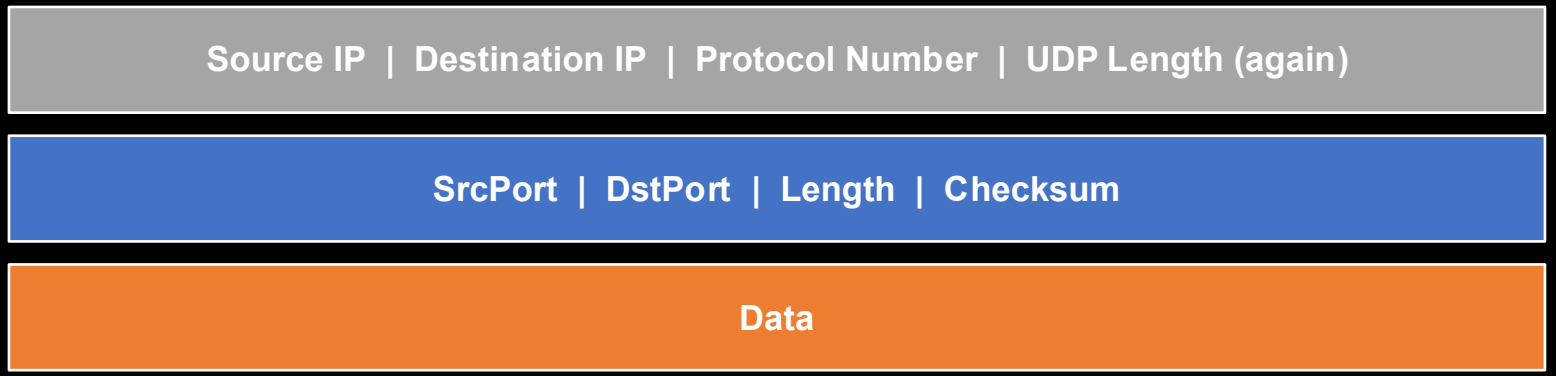
- O checksum cobre o cabeçalho UDP, os dados, e um pseudo-cabeçalho — campos emprestados do cabeçalho IP.
- **A resposta à pergunta anterior é: Sim, deteta isso —** porque o checksum é calculado em relação ao endereço pretendido, mesmo que esse endereço exista noutro cabeçalho (o cabeçalho IP).
- Isto é hoje em dia normalmente calculado pelas placas de rede.
- Obrigatório em IPv6, opcional em IPv4.

O Pseudo-Cabeçalho



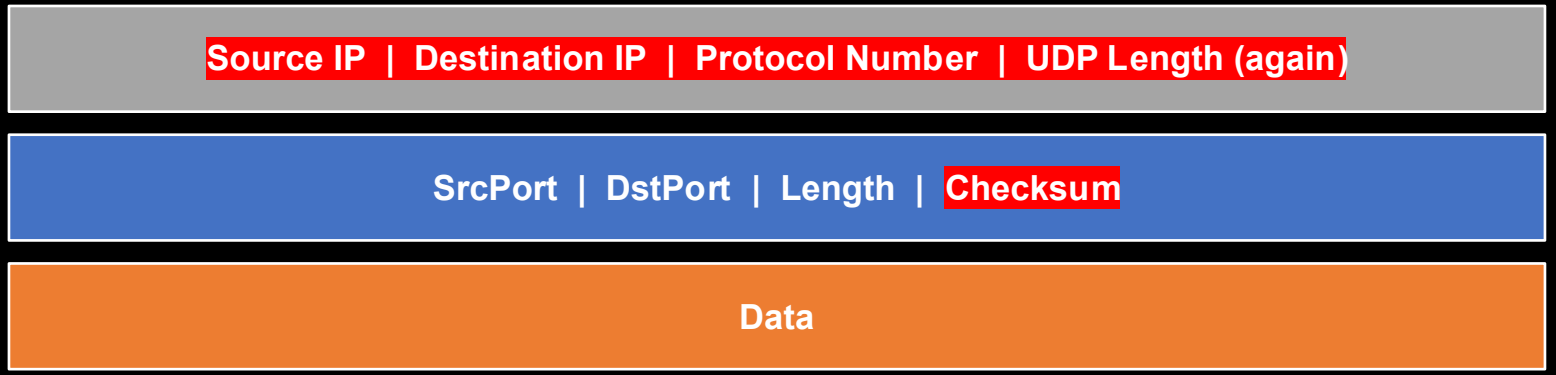
O Pseudo-Cabeçalho

emprestado do cabeçalho IP, apenas para este cálculo



O Pseudo-Cabeçalho

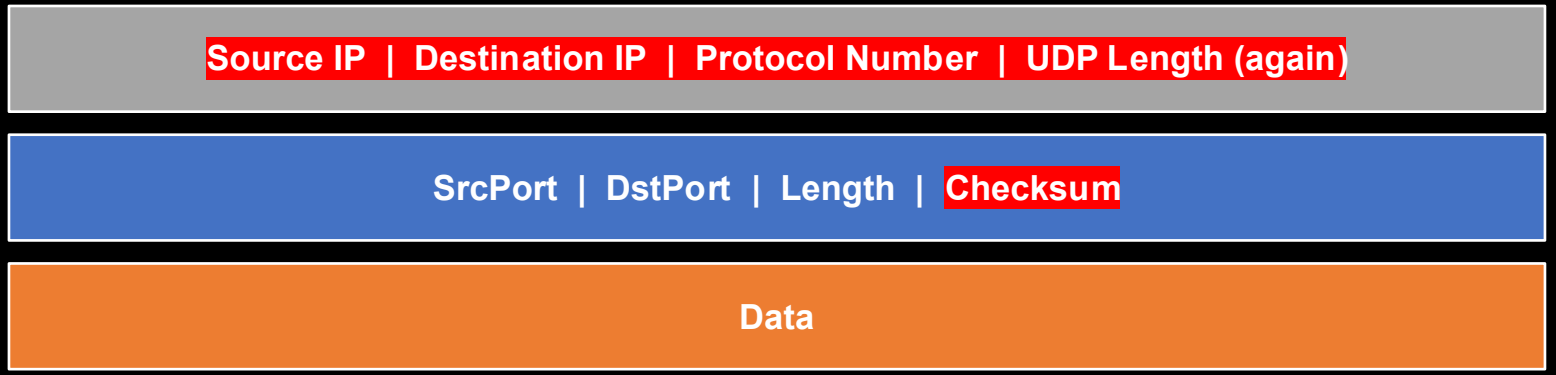
emprestado do cabeçalho IP, apenas para este cálculo



Porquê incluir endereços IP que “pertencem” a outra camada (protocolo IP)?

O Pseudo-Cabeçalho

emprestado do cabeçalho IP, apenas para este cálculo



Porquê incluir endereços IP que “pertencem” a outra camada (protocolo IP)? Para detetar entregas erradas — se o endereço de destino fosse corrompido em trânsito, o checksum (calculado em relação ao endereço pretendido) falharia também aí.

Porque Escolher UDP em vez de TCP?

- Dado tudo o que o UDP NÃO faz — sem garantias, sem ordenação, sem controlo de fluxo — porque é que alguém o escolheria em vez do TCP?

Porque Escolher UDP em vez de TCP? (1/2)

- **Sem garantias de entrega/ordenação —** um custo (para a aplicação), mas também algo positive para algumas: áudio/vídeo em tempo real não ganha nada com a retransmissão (um pacote atrasado é pior do que um perdido).

Porque Escolher UDP em vez de TCP? (1/2)

- **Sem garantias de entrega/ordenação —** um custo (para a aplicação), mas também algo positivo para algumas: áudio/vídeo em tempo real não ganha nada com a retransmissão (um pacote atrasado é pior do que um perdido).
- **Overhead mínimo —** 8 bytes vs. 20+, relevante quando as próprias mensagens são minúsculas (consultas DNS).

Porque Escolher UDP em vez de TCP? (2/2)

- **Controlo total, ao nível da aplicação, sobre a fiabilidade —** permite que a aplicação tenha controlo sobre os mecanismos de recuperação, o que lhe permite explorar a sua própria semântica, em vez de usar uma solução genérica (como a fornecida pelo TCP).

Porque Escolher UDP em vez de TCP? (2/2)

- **Controlo total, ao nível da aplicação, sobre a fiabilidade —** permite que a aplicação tenha controlo sobre os mecanismos de recuperação, o que lhe permite explorar a sua própria semântica, em vez de usar uma solução genérica (como a fornecida pelo TCP).
- **O multicast requer UDP.** O TCP é orientado a ligação e ponto-a-ponto — não consegue endereçar um grupo. A entrega um-para-muitos tem de ser construída sobre um protocolo sem ligação.

Conclusão: Porque Ainda Usamos UDP

- **Evitar o overhead do TCP —** sem estabelecimento de ligação, sem campos de cabeçalho desnecessários.
- **Dar controlo total à aplicação —** construir exatamente o comportamento de fiabilidade de que precisa.
- **Algumas necessidades das aplicações são prejudicadas por mecanismos de fiabilidade —** por exemplo streaming de vídeo, onde um pacote atrasado é pior do que um perdido.
- **O único protocolo de transporte que suporta comunicação ponto-multiponto.** Esta é uma das razões pela qual o Projeto 1 usa UDP — não é apenas uma preferência de desempenho.

Uma Pergunta Mais Difícil

- Porque é que o TCP, mas não o UDP, consegue distinguir múltiplas ligações simultâneas que chegam à mesma porta do servidor?

Uma Pergunta Mais Difícil

- Porque é que o TCP, mas não o UDP, consegue distinguir múltiplas ligações simultâneas que chegam à mesma porta do servidor?
- **Vamos responder a isto mais tarde —** Quando abordarmos o modelo de ligação do TCP. Por agora: o UDP não tem qualquer noção de “ligação” — cada datagrama é desmultiplexado (individualmente) apenas pela porta de destino, sem qualquer estado que ligue uma sequência de datagramas entre si.

Mais Uma Pergunta

- O UDP não dá absolutamente nenhuma garantia de entrega. E se uma aplicação precisar mesmo de alguma?

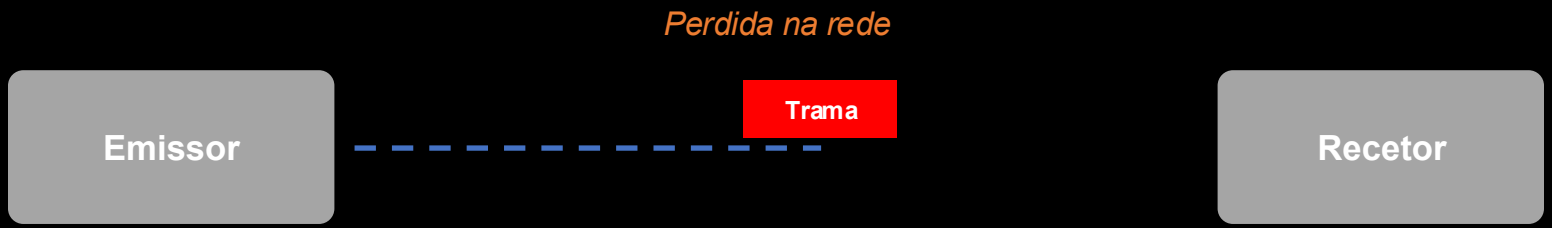
Mais Uma Pergunta

- O UDP não dá absolutamente nenhuma garantia de entrega. E se uma aplicação precisar mesmo de alguma?
- Por exemplo, imaginem que estamos a transferir um ficheiro sobre UDP e que alguns pacotes são perdidos pela rede… isso não é nada bom já que – para ficheiros maiores que não cabem num único pacote – se iria perder parte do conteúdo do ficheiro (como vão ver no primeiro projeto).

O Problema, Sem Solução Nenhuma

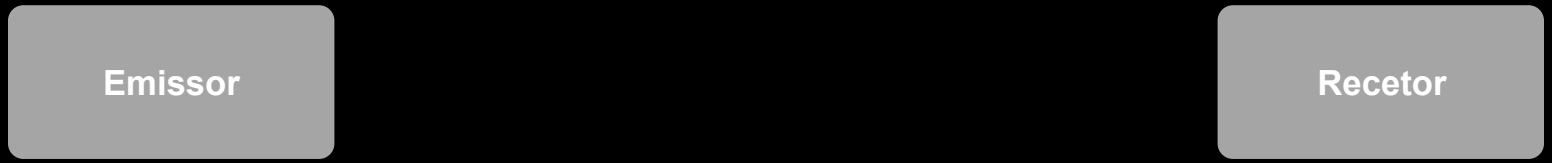


O Problema, Sem Solução Nenhuma



O Problema, Sem Solução Nenhuma

O recetor nunca a recebe.
O emissor nunca fica a saber.



O silêncio é indistinguível do sucesso — o UDP por si só não consegue distinguir os dois casos.

Então o Que É Que Precisamos?

- No mínimo: uma forma de o emissor descobrir que uma trama não chegou, e uma forma de tentar outra vez. É essa a ideia por trás do uso de confirmações (acknowledgments), temporizadores (timeouts), e retransmissões, geralmente chamado: **Automatic Repeat Request** (ou simplesmente **ARQ**).

Um Pouco de História

- **Mais antigo do que a Internet como a conhecemos:** o ARQ já era uma prática corrente em protocolos da camada de ligação de dados como o SDLC da IBM e o HDLC da ITU nos anos 1970, e no próprio protocolo de ligação IMP-a-IMP da ARPANET.
- **E não é só em ligações físicas:** o Wi-Fi (802.11) corre o seu próprio ARQ ao nível da ligação — toda a trama que envias por Wi-Fi é **stop-and-wait** reconhecida, de forma invisível, antes de o TCP ou o UDP poderem suspeitar de uma perda.

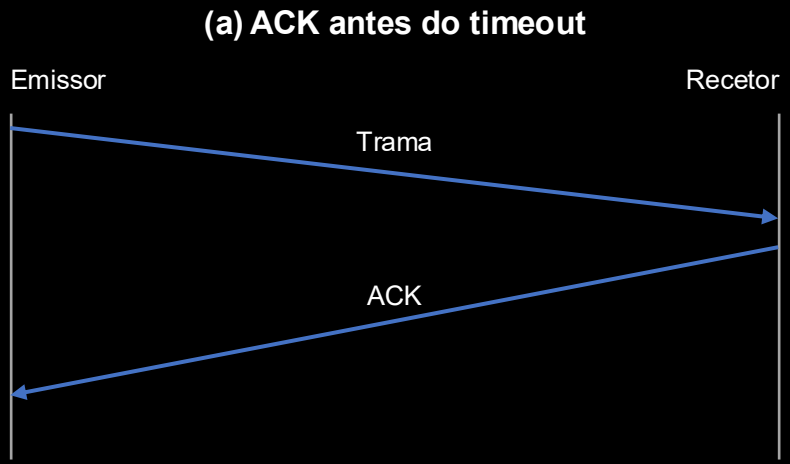
Automatic Repeat Request (ARQ)

Conclusão:
Uma estratégia abstrata para lidar com o facto de que os pacotes na rede se podem perder; a ideia é combinar confirmações, temporizadores, e retransmissões. Este padrão tem um nome — Automatic Repeat reQuest (ARQ). Tem décadas de existência, e vamos explorar como implementar uma solução destas para a própria camada de fiabilidade do Projeto 1.

Stop-and-Wait

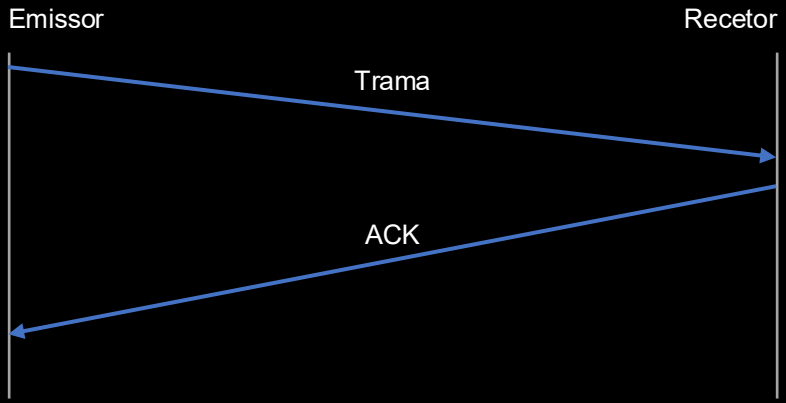
- O esquema de ARQ mais simples: enviar uma trama pela rede, esperar por um ACK antes de enviar a seguinte.
- Se nenhum ACK chegar dentro de um timeout, retransmite a última trama enviada.

Quatro Cenários

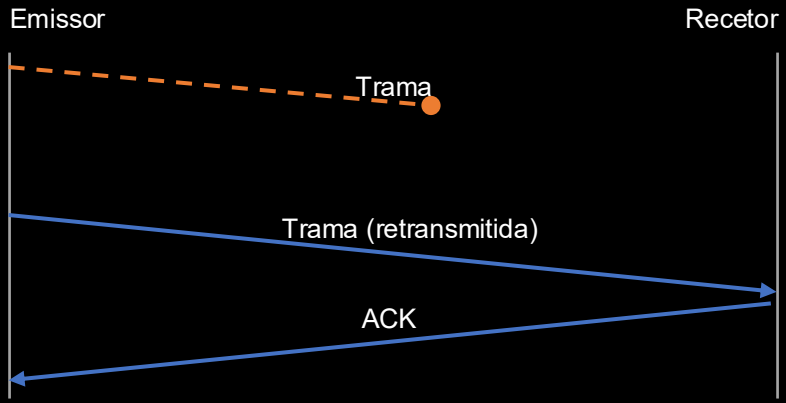


Quatro Cenários

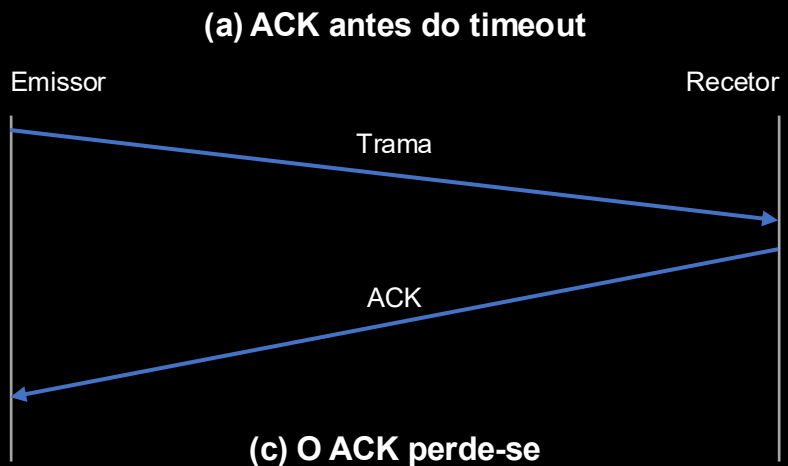
(a) ACK antes do timeout

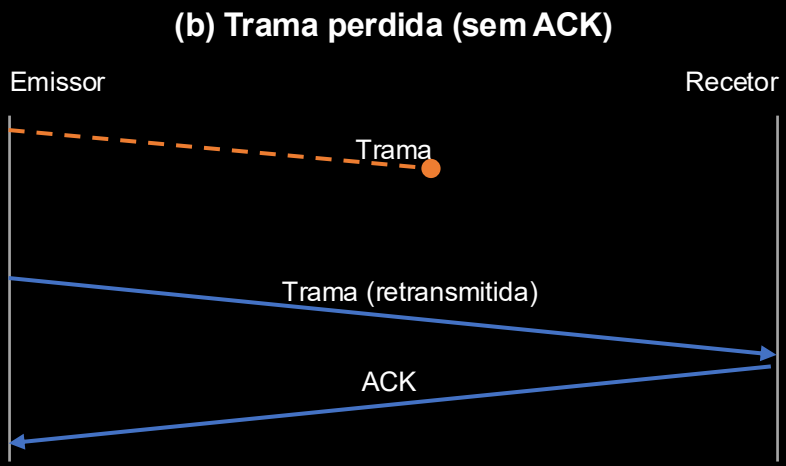


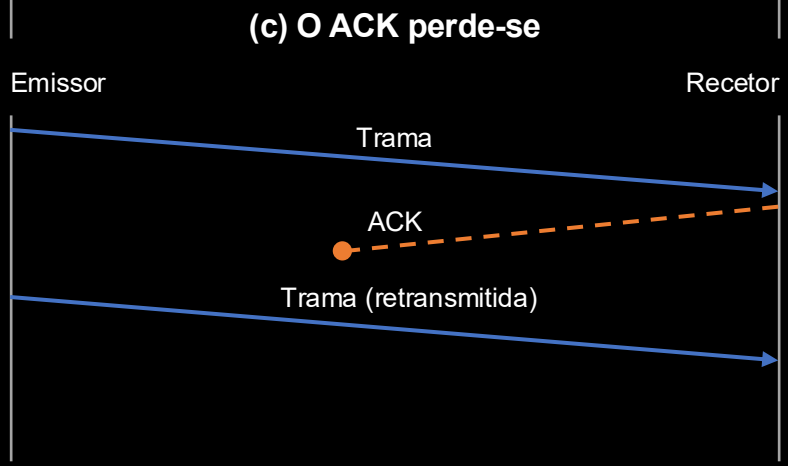
(b) Trama perdida (sem ACK)



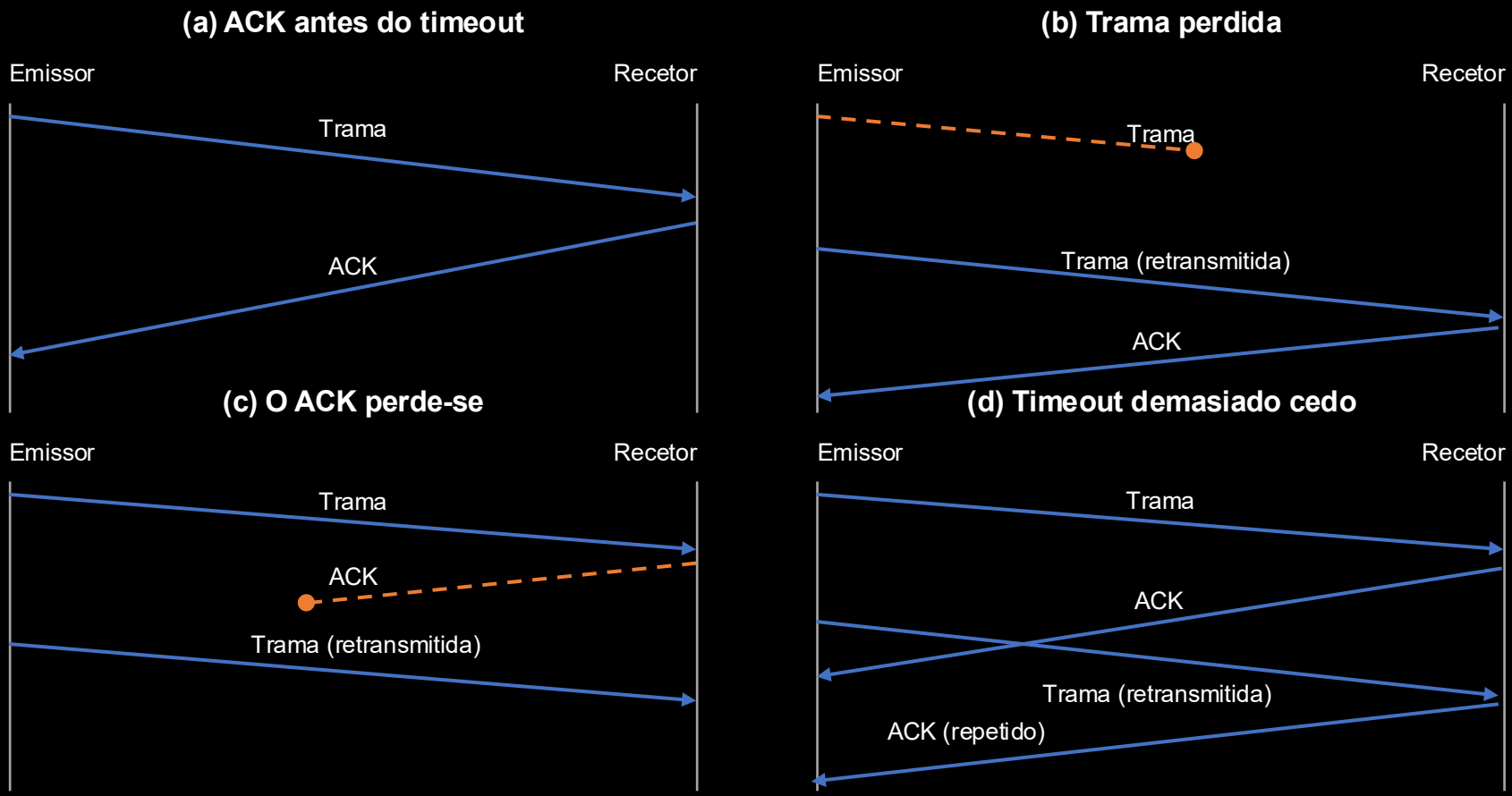
Quatro Cenários







Quatro Cenários



A Subtileza da Trama Duplicada

- Nos cenários (c) e (d), o emissor retransmite uma trama que o recetor já tem. O que é que pode correr mal para o recetor?

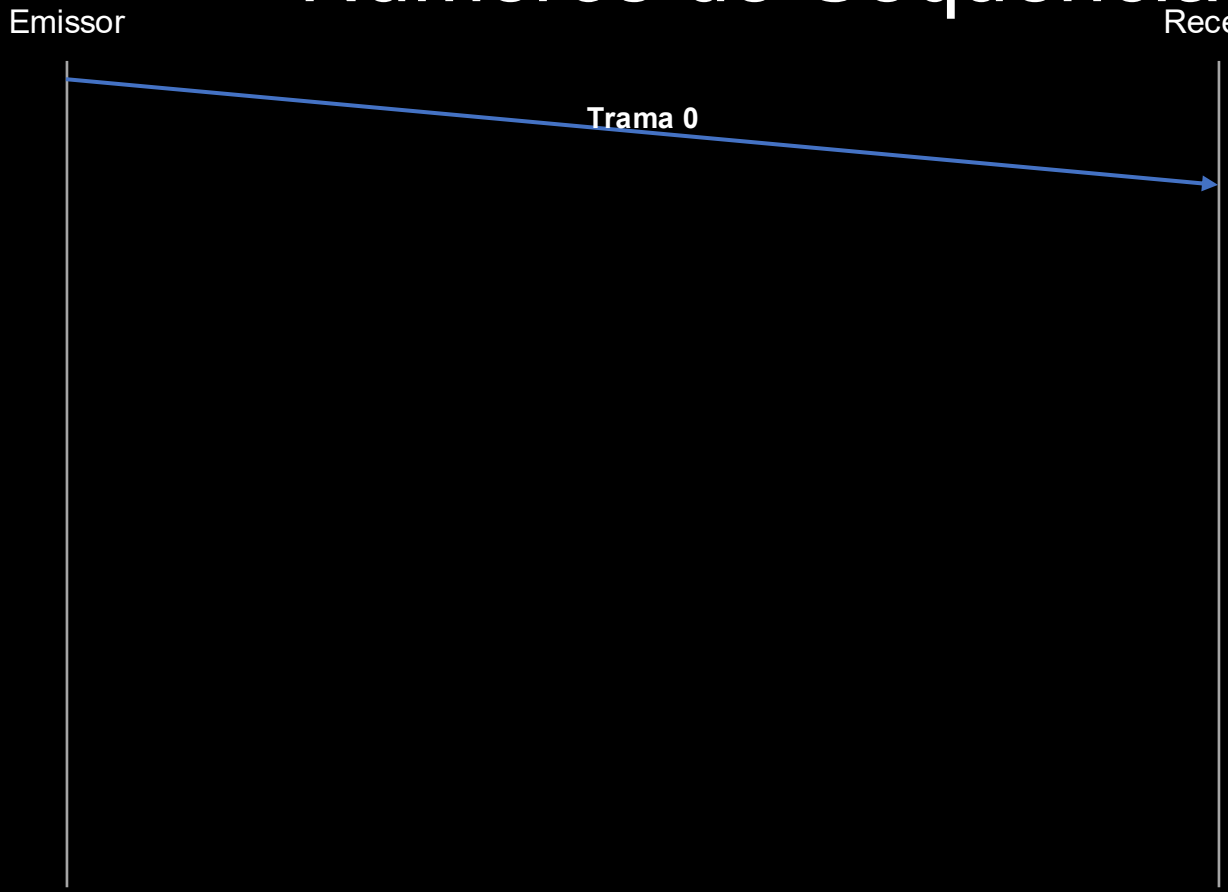
A Subtileza da Trama Duplicada

- Nos cenários (c) e (d), o emissor retransmite uma trama que o recetor já tem. O que é que pode correr mal para o recetor?
- Pode ver um duplicado de uma trama que já entregou — e não conseguir distinguir de uma trama genuinamente nova.

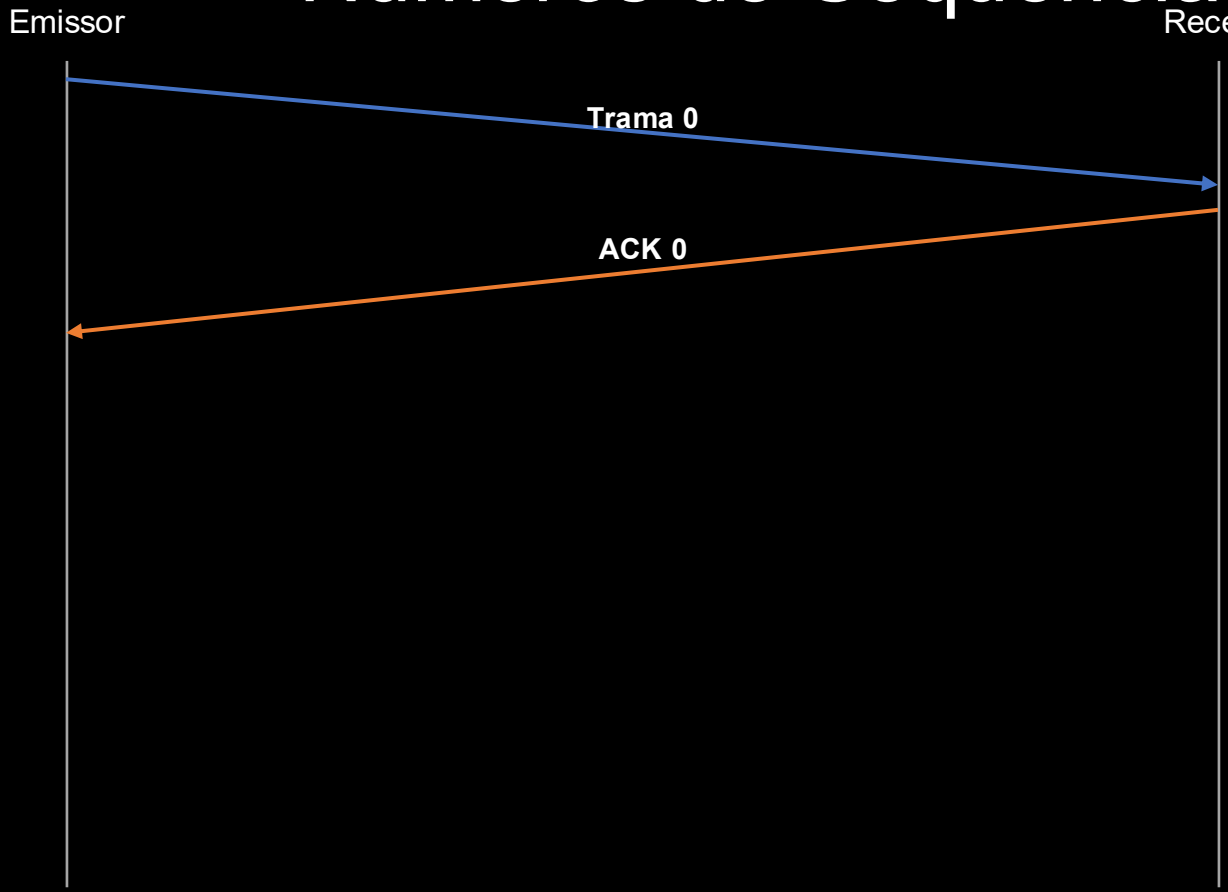
A Subtileza da Trama Duplicada

- Nos cenários (c) e (d), o emissor retransmite uma trama que o recetor já tem. O que é que pode correr mal para o recetor?
- Pode ver um duplicado de uma trama que já entregou — e não conseguir distinguir de uma trama genuinamente nova.
- **Solução: um número de sequência de 1 bit,** alternando entre 0/1 a cada nova trama.
- Um espaço de 1 bit é suficiente porque o Stop-and-Wait nunca tem mais do que uma trama em trânsito de cada vez — apenas duas “encarnações” a distinguir, sempre.

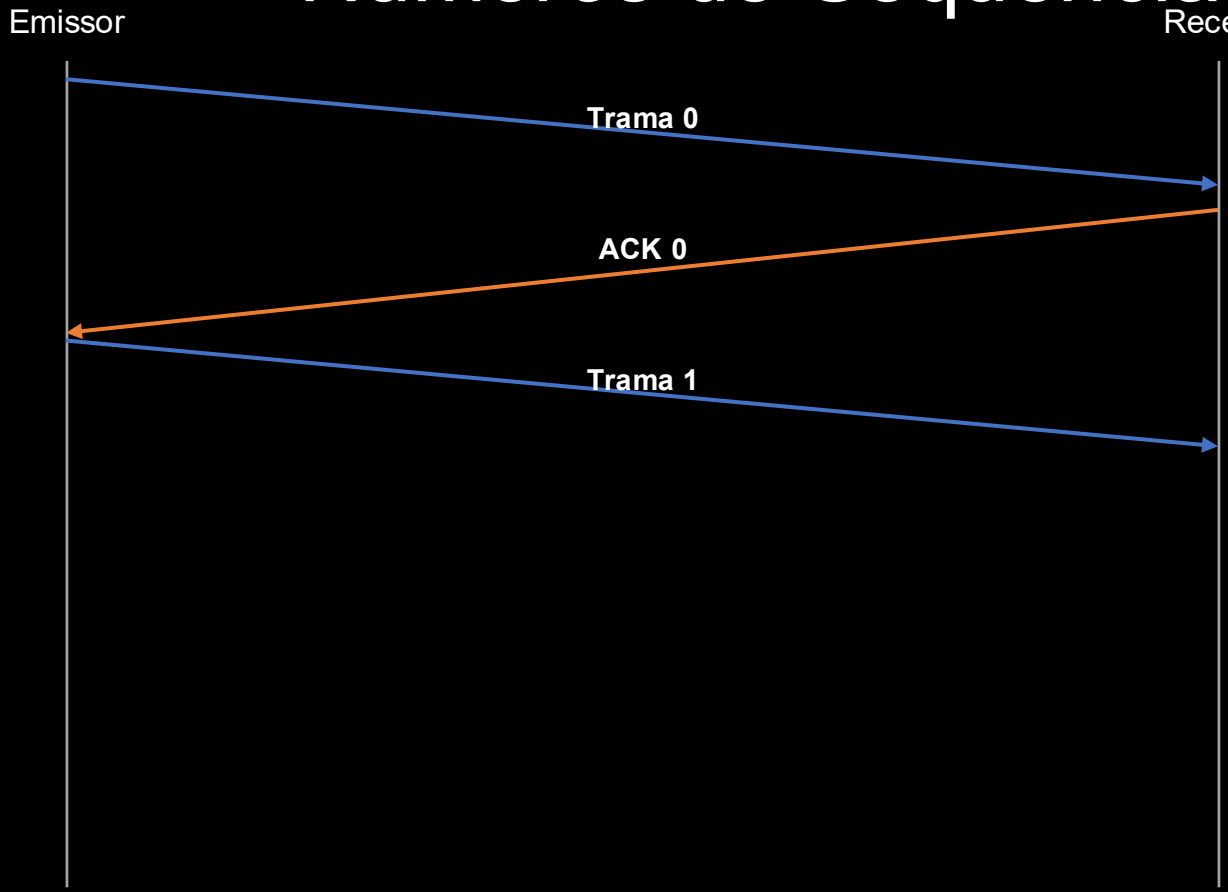
Números de Sequência de 1 Bit



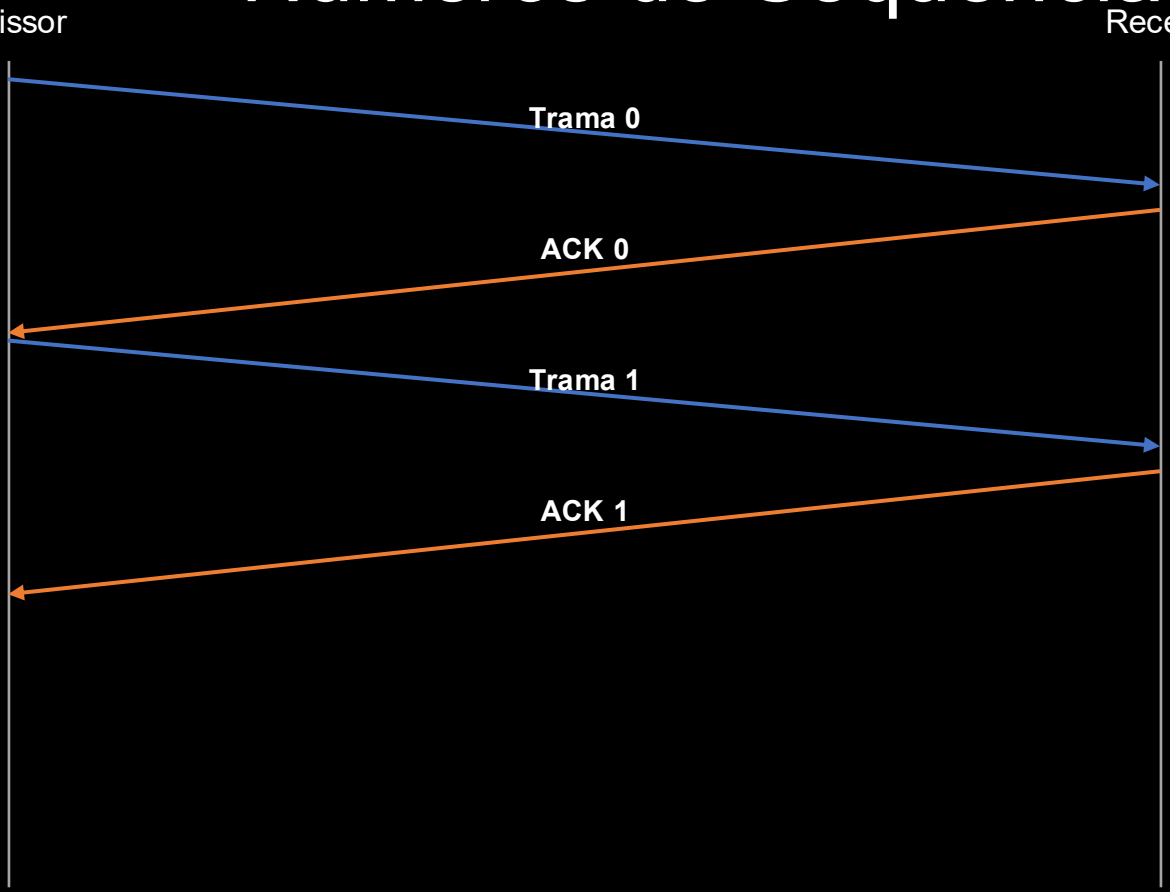
Números de Sequência de 1 Bit



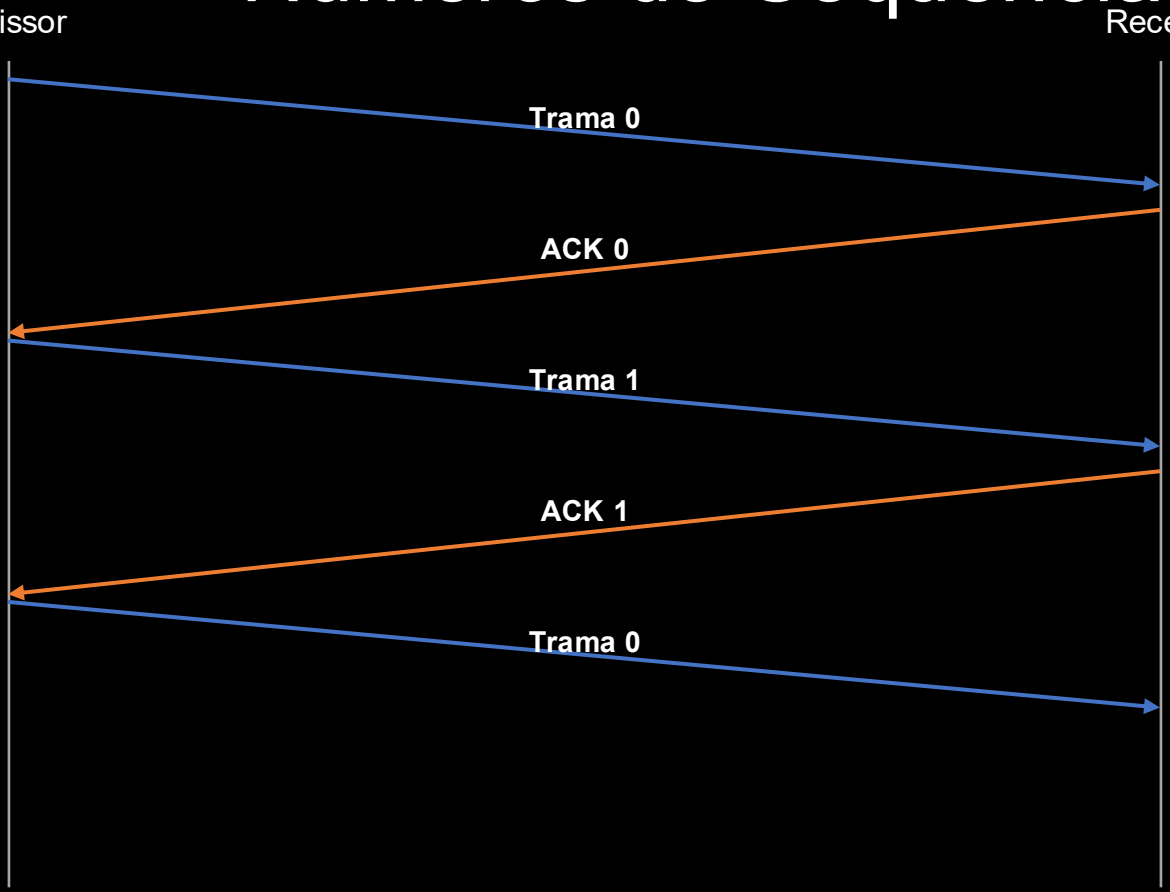
Números de Sequência de 1 Bit



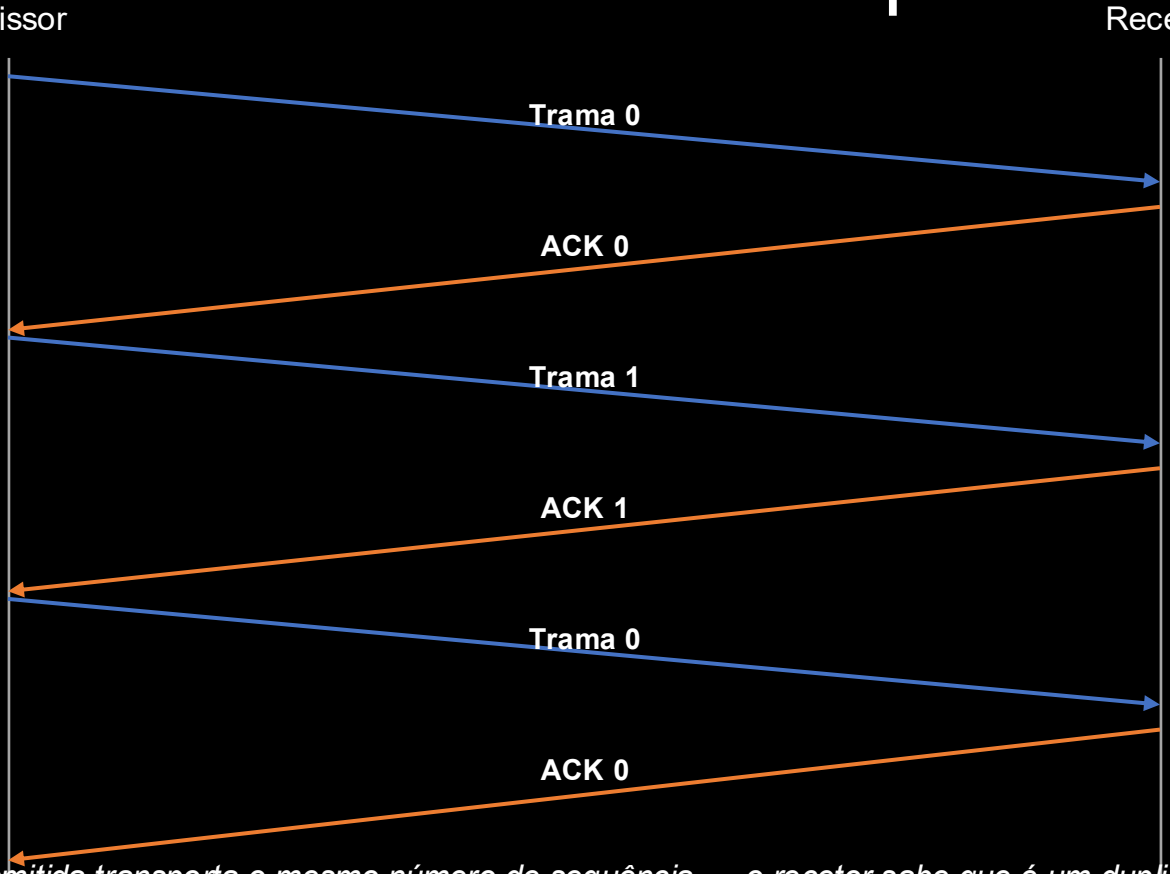
Números de Sequência de 1 Bit



Números de Sequência de 1 Bit



Números de Sequência de 1 Bit



A Trama 0 retransmitida transporta o mesmo número de sequência — o recetor sabe que é um duplicado e descarta-a (mas mesmo assim envia o ACK).

O Problema de Eficiência do Stop-and-Wait

- Na semana passada discutimos o produto do atraso × largura de banda: o Stop-and-Wait só permite uma trama do emissor em trânsito por RTT, por maior que esse produto seja.

Exemplo

- Uma ligação de 1.5 Mbps, com 45 ms de RTT.

Exemplo

- Uma ligação de 1.5 Mbps, com 45 ms de RTT.
- Atraso × Largura de Banda ≈ 67.5 Kb (≈ 8 KB).

Exemplo

- Uma ligação de 1.5 Mbps, com 45 ms de RTT.
- Atraso × Largura de Banda ≈ 67.5 Kb (≈ 8 KB).
- Com tramas de 1 KB, uma trama por RTT:

Exemplo

- Uma ligação de 1.5 Mbps, com 45 ms de RTT.
- Atraso × Largura de Banda ≈ 67.5 Kb (≈ 8 KB).
- Com tramas de 1 KB, uma trama por RTT:
- Bits-Per-Frame / Time-Per-Frame = 1024 × 8 / 0.045 ≈ 182 kbps

Exemplo

- Uma ligação de 1.5 Mbps, com 45 ms de RTT.
- Atraso × Largura de Banda ≈ 67.5 Kb (≈ 8 KB).
- Com tramas de 1 KB, uma trama por RTT:
- Bits-Per-Frame / Time-Per-Frame = 1024 × 8 / 0.045 ≈ 182 kbps
- ≈ 1/8 da capacidade real da ligação.

Capacidade Ociosa

Conclusão:
para usar a ligação do exemplo anterior na totalidade, precisaríamos de manter ~8 tramas em trânsito ao mesmo tempo.

(Evidentemente, links com características diferentes resultam num número diferente de tramas — o número de tramas que precisam de estar em trânsito para garantir o uso total da ligação cresce com o produto do atraso × largura de banda).

Desafio

- Acabámos de mostrar que o Stop-and-Wait desperdiça largura de banda porque mantém no máximo uma trama em trânsito do lado do emissor (quando a ligação poderia transferir mais tramas num único RTT). Mantendo a mesma maquinaria de ACK-e-timeout — qual é a única coisa que mudariam?

Desafio

- Acabámos de mostrar que o Stop-and-Wait desperdiça largura de banda porque mantém no máximo uma trama em trânsito do lado do emissor (quando a ligação poderia transferir mais tramas num único RTT). Mantendo a mesma maquinaria de ACK-e-timeout — qual é a única coisa que mudariam?
- **Guardem essa resposta —** a próxima aula começa exatamente aí.

Sumário (1/2)

- Os protocolos de transporte fazem a ponte entre o que as aplicações querem e o que uma rede best-effort oferece.
- O UDP acrescenta exatamente uma coisa a esse serviço best-effort: multiplexagem, através de portas.
- Opcionalmente (mas de forma comum), o UDP também fornece um mecanismo para detetar erros/corrupção de dados — convertendo esse erro num pacote perdido.
- O cabeçalho UDP tem 8 bytes fixos: SrcPort, DstPort, Length, Checksum.

Sumário (2/2)

- A encapsulação aninha cabeçalhos na saída (UDP, depois IP); a desencapsulação remove-os pela ordem inversa na entrada.
- O checksum (opcional em IPv4) cobre também um pseudo-cabeçalho, para detetar entregas erradas.
- O UDP troca garantias por controlo, baixo overhead, e suporte para multicast.
- Um padrão comum para lidar com a perda de pacotes na rede é usar reconhecimentos, temporizadores, e retransmissões: geralmente conhecido como Automatic Repeat Request (ARQ).
- O Stop-and-Wait é um algoritmo para materializar o ARQ, em que cada pacote tem de ser reconhecido antes de se enviar o pacote seguinte. Isto pode levar a desperdício de largura de banda.

Bibliografia

- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Chapter 5, Section 5.1.
- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Chapter 1, Section 1.3 — revisão da Aula 2.
- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Chapter 2, Section 2.5.1 — Stop-and-Wait.

©2026 NOVA FCT